



UNIVERSITY OF
WATERLOO

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ECE-124 LAB MANUAL

V3.0
LAB 1

Course: ECE-124 Digital Circuits and Systems
(with Verilog)

1 Introduction and ECE-124 Labs Outline

ECE-124 is an introductory course on Digital Logic Design and Implementation

Following the Lab Manual is important but please note that:

FURTHER DETAILS FOR THE LAB SESSION FPGA DESIGN WORK IS PROVIDED DURING THE LAB SESSION. ATTENDANCE IS THEREFORE **REQUIRED**.

Each laboratory experiment has several parts:

1. A three-hour lab session, which you can use to develop your Team design. Help is available.
2. A Demonstration of your Team Design for the project. (done at the NEXT lab session)
3. A short minute quiz on Lab subject matter is to be completed by each student. (done at the NEXT lab session)

In the Lab:

1. **Absolutely - no food or drink in the on the workstations in the laboratory.**
2. Do not block the doors or windows open. The after-hours access will cease if the rules are not followed by the students.

Each workstation in the ECE 124 lab has:

1. University of Waterloo LogicalStep Board housing an Altera MAX10 Field Programmable Gate Array (FPGA) chip and various peripheral components
2. Intel (Altera) Quartus-Prime FPGA Design v18.1 Software

EACH STUDENT IS EXPECTED TO SECURE THEIR OWN LAB WORK.

AFTER EACH LAB SESSION, ZIP YOUR LAB PROJECT FOLDER AND COPY IT TO THE N: DRIVE FOR BACKUP.



We are going to be briefly covering the design of the board platform that will be used to prove the implementation of the FPGA designs during the course. This board platform is the University of Waterloo - LogicalStep board. Figure 1 is a photo of the U of W LogicalStep board that will be used for your experiments in the course.

Figure 1 - LogicalStep Board

Below is a sketch of the external FPGA designer resources (peripherals) that can be used for various FPGA design projects.

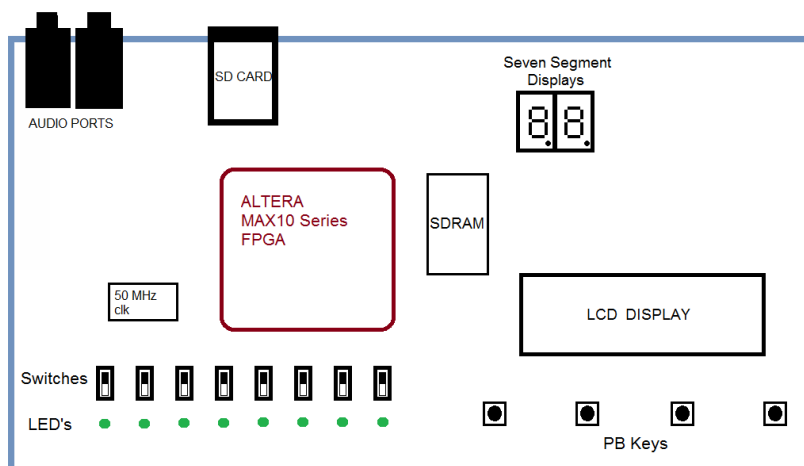


Figure 2 - LogicalStep Block Diagram

1.1 LogicalStep FPGA Board User Peripherals

The LogicalStep board is equipped with some peripherals that can be used to create various applications. These peripherals are things such as a clock source, switches and Push-Button Key inputs, output LED indicators, SDRAM, SD Card (like a mini-disk) and audio electronics etc.

For the ECE-124 labs, we are only using the clock source, switches and push buttons for supplying logic inputs and for outputs; we will use the LED's and the Seven-Segment Displays.

LEDs are electronic components, which can emit light with much greater efficiency than incandescent lamps. Specific pins on the FPGA are connected on the LogicalStep board to drive these LED's.

Other LED indicators on the board are the Seven-Segment displays (so called because it is an arrangement of 7 bar segments). They are arranged in such a way that the numbers 0 to 9, and letters A to F, can be displayed.

Seven segments can uniquely display numbers and that is why they are used for many calculator or electronic displays. To drive these displays a Seven-Segment decoder (inside the FPGA) is usually used to convert a 4-bit binary number to a Seven-Segment LED pattern so that a person can see a number or letter instead of trying to interpret the original 4-bit binary number.

There are also eight slide switch inputs and four push-button key inputs. A 50 MHz clock is connected to the FPGA to drive logic processing.

1.2 Field Programmable Gate Array (FPGA) Technology

FPGA technology has been around since the mid 1980's. It has gradually grown in complexity and capability and is now a ubiquitous technology used in the workplace. For a typical FPGA there are two prime components involved within its architecture. These are the Logic Elements (LE's) and the interconnect resources. Both items are completely configurable after the FPGA device is powered. The Logic Elements are used to implement gate-level logic, and the interconnect section is used to connect the various LE's in some arrangement. The design files that are downloaded into the device are developed on an external software-based design platform (such as Altera Quartus Prime). When a designer has processed the design in Quartus, its load file can be sent into the FPGA.

But one might ask "How does a FPGA download file become functioning hardware?".

In the sketch shown below all the logic functions and interconnects that are in an FPGA device are controlled by configuration memory cells located in the FPGA. These memory cells directly control the activation or deactivation of switch transistors. These transistors are placed in the FPGA to control logic lookup-tables (inside the LE blocks) and interconnect path link connections.

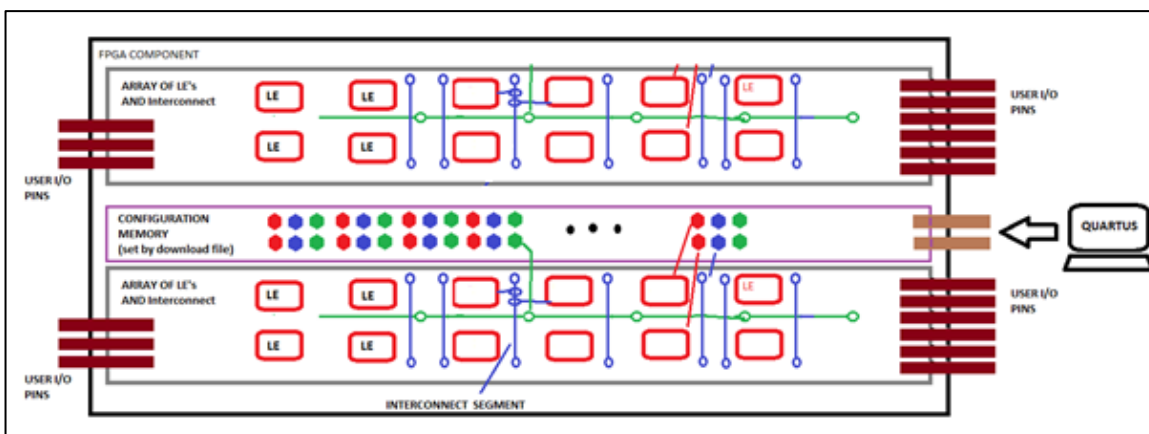


Figure 3 - Typical FPGA Configuration Memory

Thus, a download file can be used to implement any logic hardware function or functions within the FPGA. Since the FPGA can be reconfigured, it can implement different hardware functions by simply having different download files sent to it.

The external I/O connections are usually fixed on a board design however and they must be kept in mind when considering new FPGA designs. FPGAs are different from microprocessors or microcontrollers because the designer is able to change the **hardware design** of the logic.

The FPGA device that is used on the LogicalStep board that you will be using is a current technology FPGA platform. Although it is one of the smaller FPGA's offered in the marketplace it can still pack a lot of functionality into the device's 8K LE's and 144 pins. This FPGA offers many other kinds of internal resources (DSP', RAM, PLL's) but they are beyond the scope of the Lab course.

1.3 FPGA Design Software

This course will be using the Quartus Prime set of tools to develop the FPGA designs and it is a full FPGA design software suite from Altera (now Intel). It aids the designer through the different stages of describing the hardware design and targeting it for a certain Altera FPGA chip. The typical development process uses the following stages:

- Design Entry:
 - Schematic Entry: by connecting blocks of ranging complexity. It can be used to interconnect simple components such as simple logic gates or to interconnect previously created hardware blocks
 - Hardware Description Language: such as VHDL or Verilog (we use Verilog)
- Design Analysis and Synthesis Compilation:
 - An HDL or schematic file is analyzed and, if no errors are present, it is mapped into a number of logic gate equivalents and the connections between them. The synthesized file can be used for functional simulations.
- FUNCTIONAL Simulation:
 - Functional Simulation: This is used to verify the logic functionality of the design before further processing steps are attempted. Stimulus is applied and the waveform results follow the functionality of the synthesized design. Any functional errors are observed in these simulations and then they are fixed in the original design. The design must be recompiled for a new synthesis and subsequent simulation testing.
- Design Fitting Compilation:
 - After the design is functionally proven through functional simulation, it is further processed for a specific FPGA device. This compilation involves the organizing of the synthesized gates into the Logic Element resources of the FPGA (Placement phase) and then determines the routing required between the Logic Elements (Routing Phase). The placement and routing details yield timing information about the "FITTED" design. This information can be used for Timing simulations.

- Timing Simulation:
 - This is usually required for higher performance or higher density designs that have been fitted. It gives an accurate indication of how fast the circuit can run and how much timing margin is available for various operating conditions. Timing violation errors are fixed in the design. This kind of simulation will not be exercised in this course.
- Design Assembly Compilation:
 - An FPGA load file is produced so that it can be downloaded to the FPGA chip.
- Programming the FPGA:
 - The circuit can be physically tested afterwards by applying inputs and observing the outputs.

1.4 ECE-124 Lab Sessions Outline:

Lab Sessions:

1. Session 1: Lab Procedures; Lab1- Design Entry Methods Using Intel (Altera) Quartus Prime Tools (3 hrs)
2. Session 2: Demo from Lab1; Lab2- Verilog for Combinational Circuits PART 1 (3 hrs) – Simple ALU Design
3. Session 3: Demo from Lab2; Lab3- Verilog for Combinational Circuits PART 2 (3 hrs) – Energy Monitor
4. Session 4: Demo from Lab3; Lab4- Verilog for Sequential Circuits (3 hrs)- Registers, State Machine Designs
5. Session 5: Demo from Lab4.

Each Lab WorkStation is made for a group of two. Find a lab partner before or during the first Lab.

2 Lab 1 – Design Entry Using Altera Quartus-II

The goal of this lab session is primarily to gain experience with the Quartus Prime FPGA Design Environment. Lab1 will go through two design entry methods and some simulation steps for design testing. Later the design will be processed for the programming of an FPGA on the LogicalStep board to observe how the logic circuit works in hardware.

2.1 Lab1 Intended Learning Outcomes

By the end of this lab, students should be able to:

- 1) **CONSTRUCT** a digital circuit in both schematic and Verilog representations.
- 2) **VERIFY** digital circuit representations using functional simulations.
- 3) **TEST** your FPGA design on the LogicalStep FPGA Development Board

2.2 Prelab

No prelab work is necessary for LAB1.

2.3 Lab1 Outline:

Attendance will be taken. Your Team partnerships will be settled before or during LAB1 using Learn Groups.

Lab 1 is composed of the following main categories:

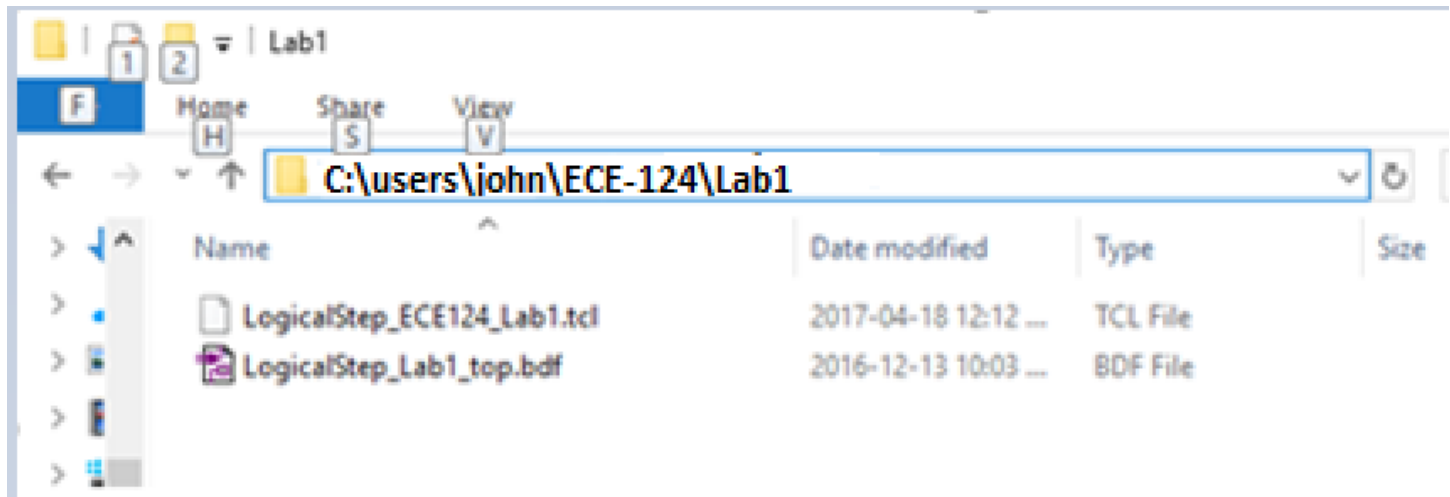
1. A brief introduction to the laboratory, its equipment and the student conduct expected during Lab sessions.
2. Learning two Design entry methods (Schematic and Verilog) for small digital circuits.
3. Running Synthesis and Simulation processes on the circuits to check that they operate as expected.
4. Compiling the circuits into files for programming the LogicalStep board to confirm that the hardware implementation functions as expected.
5. Modifying the circuit designs to provide new functionality and then test them.

2.4 Lab1 Activities

2.4.1 Starting Your Lab1 Project

To begin your Lab1 project use the Windows File Explorer to browse to your USERS folder on the workstation C: Drive. In that folder IN there, create a subfolder for the course called ECE-124.

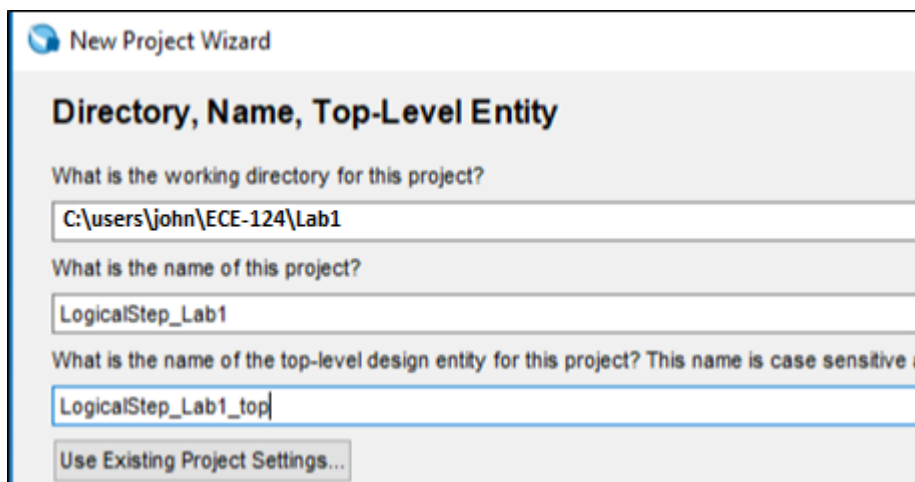
Log onto LEARN and download the Lab1 Zipped folder “Lab1.zip” and move it into the newly created **ECE-124** folder on the C: Drive. Open the zip file and extract the contents into the **ECE-124** folder. The Lab1 folder will be created during the **EXTRACTION**. An example for C: drive use is shown below:



Go to the Windows START Menu and invoke the Altera Quartus Prime Tool

Inside Quartus Prime, start the process to setup a new project (**DO NOT USE THE BLUE SCREEN METHOD TO SETUP A PROJECT**). **USE THE FOLLOWING ONLY:**

SELECT **FILE>New Project Wizard** as shown below. Click **NEXT** to go to the second slide.



MAKE SURE THAT NO SPACES ARE USED IN THE NAMES

The project parameters will now be entered as shown (**note that these fields are CASE sensitive**).

Project Folder: C:\USERS\john\ECE-124\Lab1

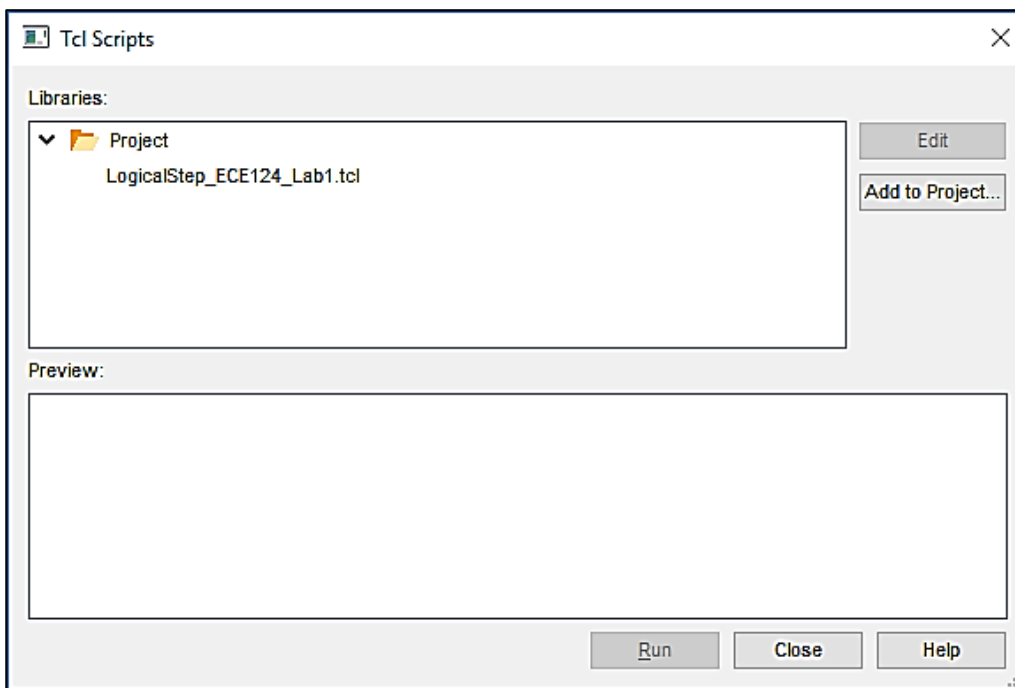
Project Name: **LogicalStep_Lab1**

Project Top Level: **LogicalStep_Lab1_top**

Then, Click **FINISH** on the Wizard Dialog Window.

******* VERY IMPORTANT NEXT STEP *******

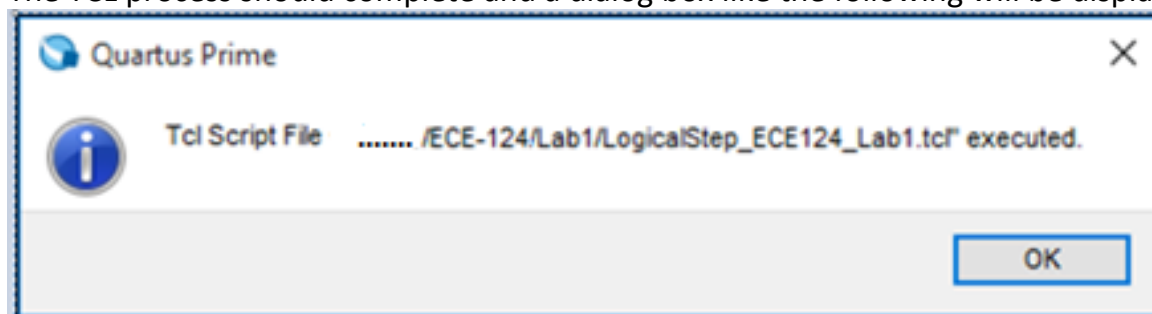
Next, in Quartus, the TCL script must be run to assign the FPGA device type and pin assignments for the FPGA being used on the LogicalStep board. These pins are reserved for connections to the LogicStep FPGA and must not be changed.



Go to the Tools TAB and SELECT **Tools>Tcl Scripts**. The dialog box (at the left) should appear.

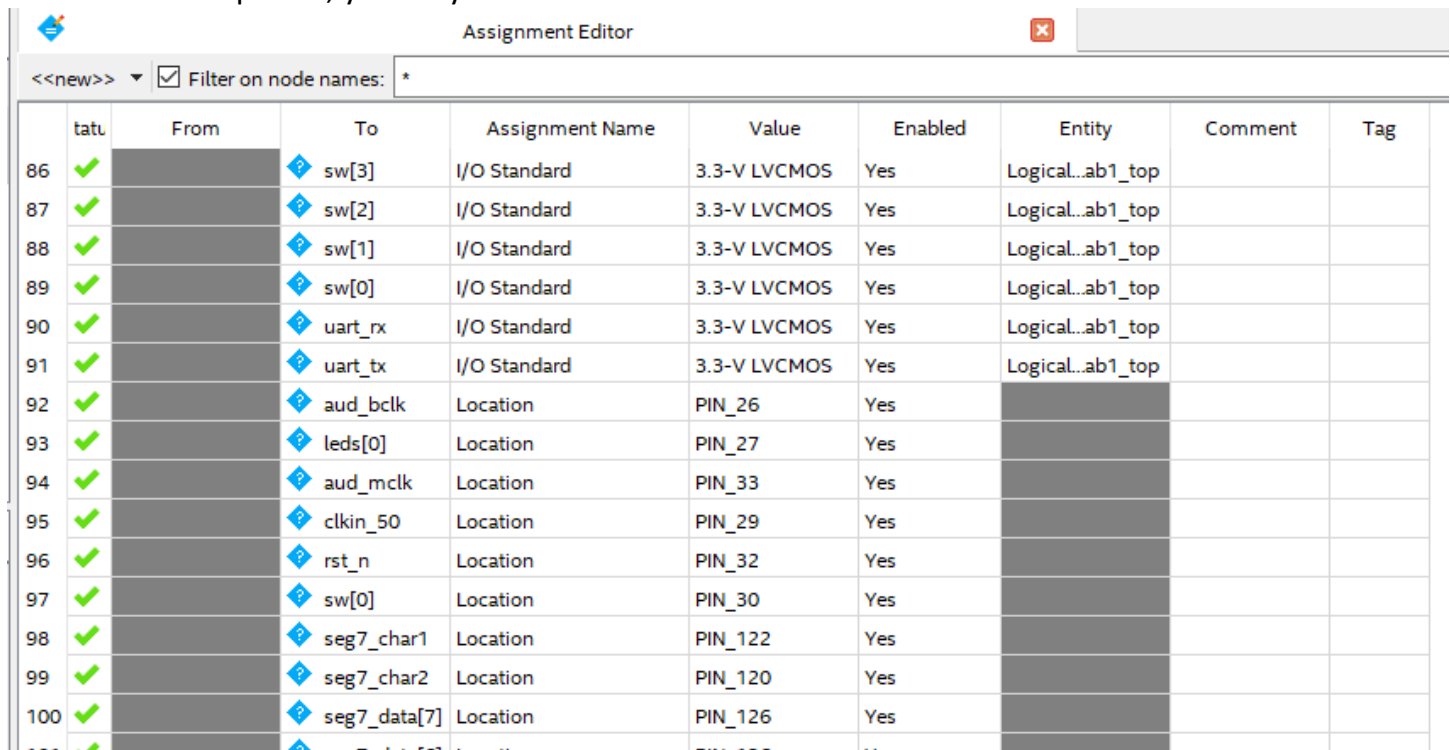
SELECT the TCL (pronounced as “tickle”) file and then click on the **RUN** button

The TCL process should complete and a dialog box like the following will be displayed.



Click **OK**. Then CLOSE the TCL Script Tool.

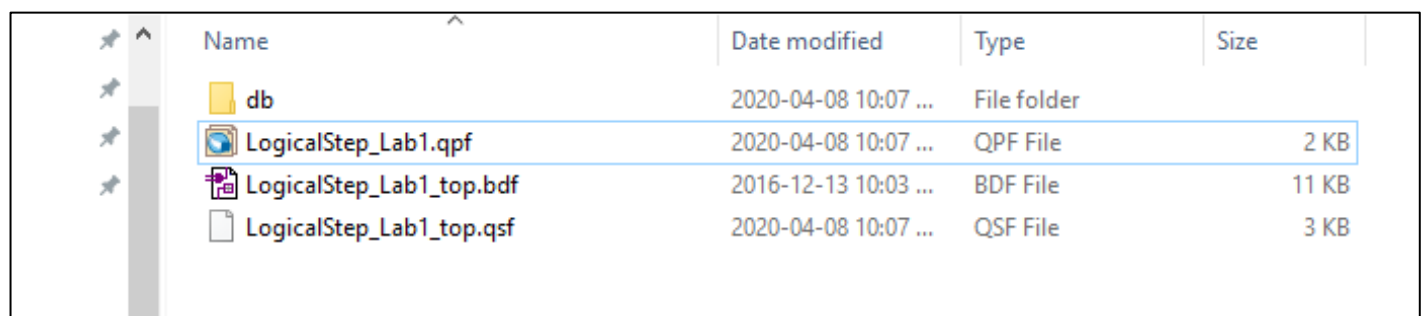
To confirm the pin assignments for the FPGA, click on the **Assignments>AssignmentEditor** function. You should see something like the following for the pin assignments. After this is completed, you may close the tcl viewer window.



	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
86	✓		sw[3]	I/O Standard	3.3-V LVCMOS	Yes	Logical...ab1_top		
87	✓		sw[2]	I/O Standard	3.3-V LVCMOS	Yes	Logical...ab1_top		
88	✓		sw[1]	I/O Standard	3.3-V LVCMOS	Yes	Logical...ab1_top		
89	✓		sw[0]	I/O Standard	3.3-V LVCMOS	Yes	Logical...ab1_top		
90	✓		uart_rx	I/O Standard	3.3-V LVCMOS	Yes	Logical...ab1_top		
91	✓		uart_tx	I/O Standard	3.3-V LVCMOS	Yes	Logical...ab1_top		
92	✓		aud_bclk	Location	PIN_26	Yes			
93	✓		leds[0]	Location	PIN_27	Yes			
94	✓		aud_mclk	Location	PIN_33	Yes			
95	✓		clkin_50	Location	PIN_29	Yes			
96	✓		rst_n	Location	PIN_32	Yes			
97	✓		sw[0]	Location	PIN_30	Yes			
98	✓		seg7_char1	Location	PIN_122	Yes			
99	✓		seg7_char2	Location	PIN_120	Yes			
100	✓		seg7_data[7]	Location	PIN_126	Yes			

If the above is not achieved, then ask the instructor for guidance.

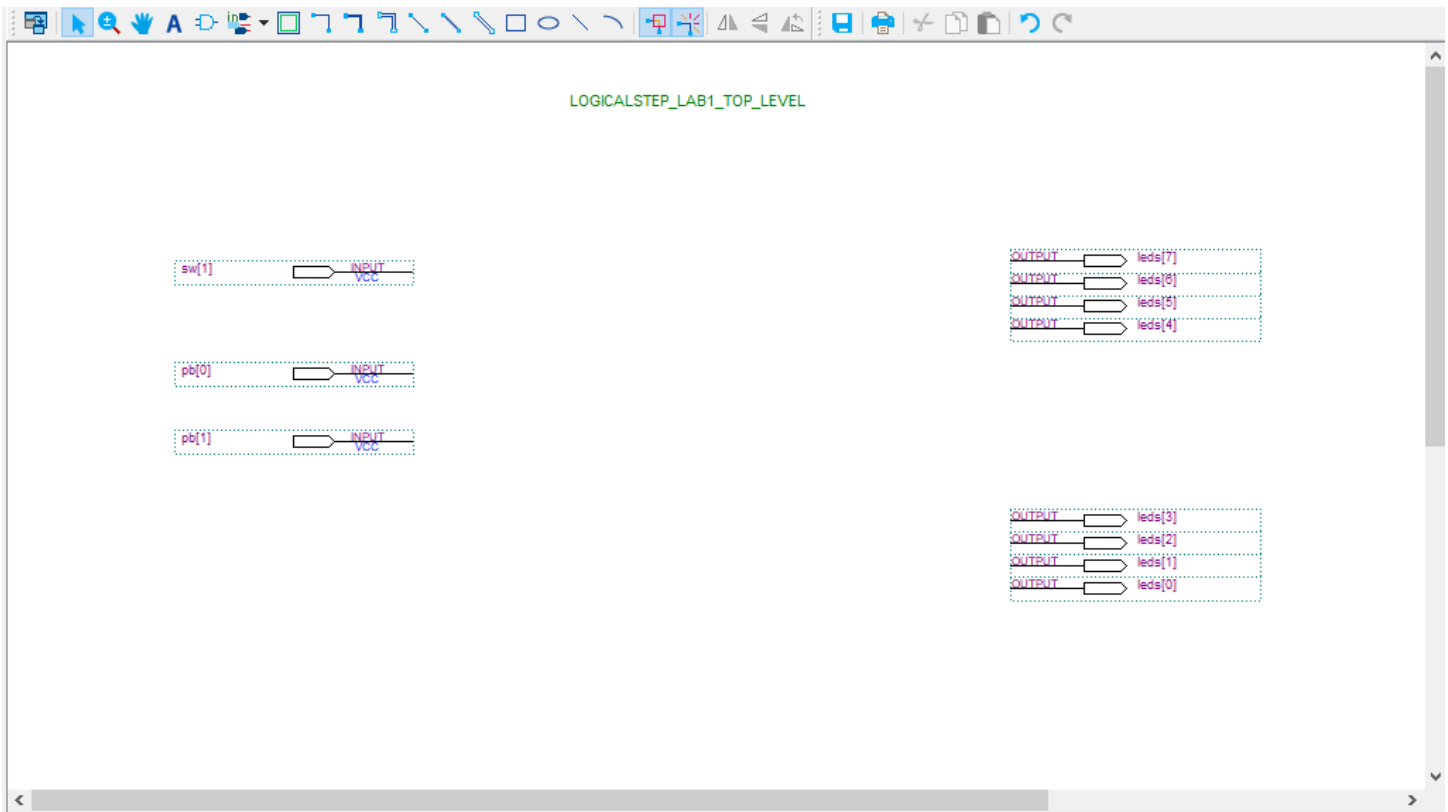
After the project setup steps have been completed you should see the following in your Lab1 project folder:



Name	Date modified	Type	Size
db	2020-04-08 10:07 ...	File folder	
LogicalStep_Lab1.qpf	2020-04-08 10:07 ...	QPF File	2 KB
LogicalStep_Lab1_top.bdf	2016-12-13 10:03 ...	BDF File	11 KB
LogicalStep_Lab1_top.qsf	2020-04-08 10:07 ...	QSF File	3 KB

Within Quartus, go to the FILE Tab and SELECT **File>Open** and then browse to the project top-level file the LogicalStep_Lab1_top.bdf file (shown above).

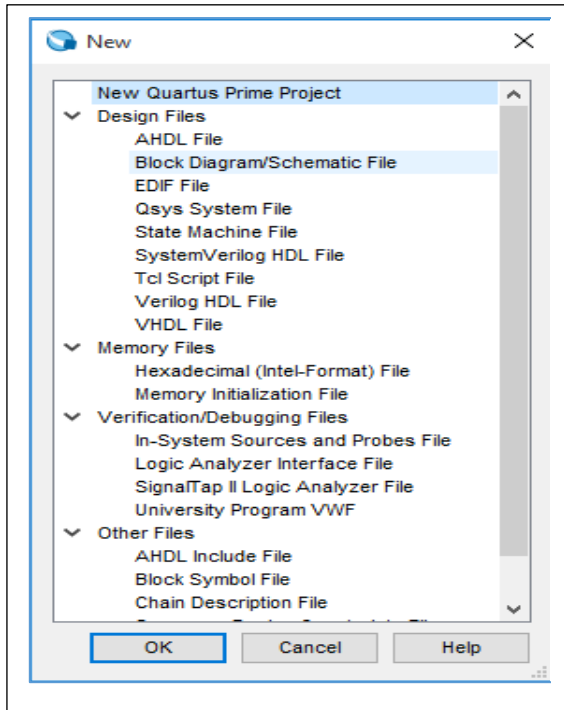
The **top-level** file is in schematic form (see below). Schematic entry methods for a simple set of gate-level functions are in this first lab.



There are three INPUT pins (pb(0), pb(1), SW(1)) on the Left Hand Side and eight OUTPUT pins (LEDS(7) to LEDS(0)) on the Right Hand Side.

NOTE: if you are having visual alignment or scaling issues on an installation of Quartus on your home computer display, a good document to refer to is:

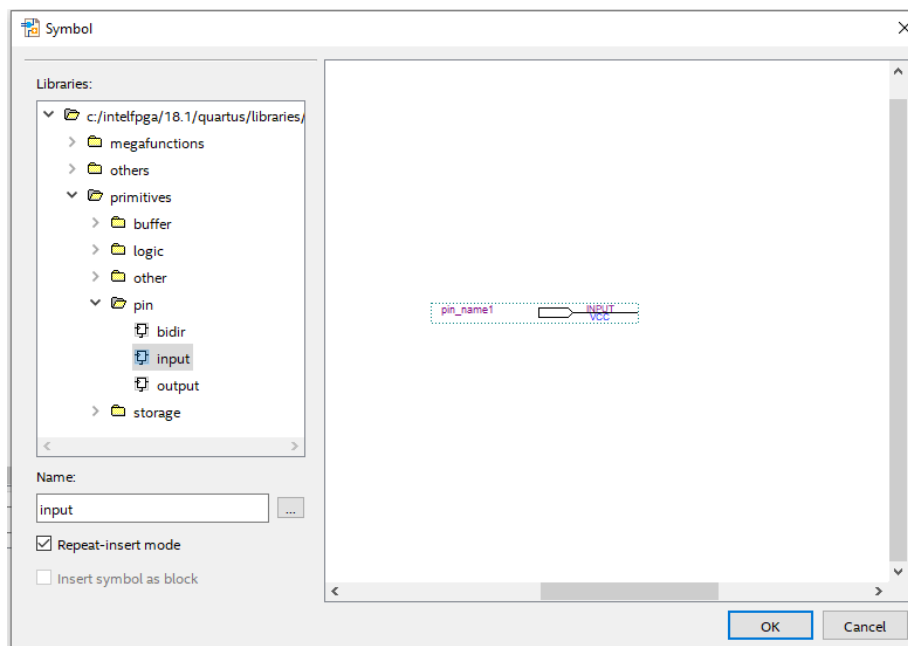
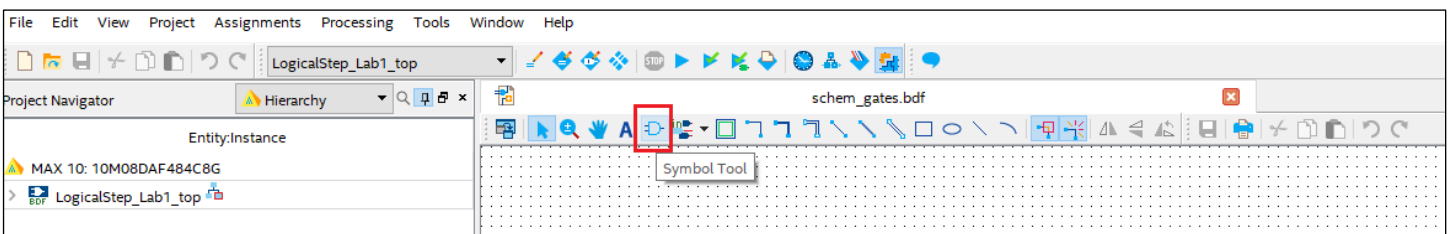
<https://eecs.blog/fix-applicationquartus-scaling-for-4k-monitor/>



Now we can start adding some design hierarchy by creating functional blocks and then installing them at the top-level design later. The first block to create will use the schematic entry format again. Go to the FILE Tab and select **File>New**. A dialog box as shown will open.

SELECT the **Block Diagram/Schematic File**. Click OK. A new blank schematic window will then open up in Quartus. After it opens, save the schematic file as “**schem_gates.bdf**” (**note the underscore used instead of a SPACE since the tools don’t like spaces in filenames**) by using the FILE Tab and selecting **File>Save As**.

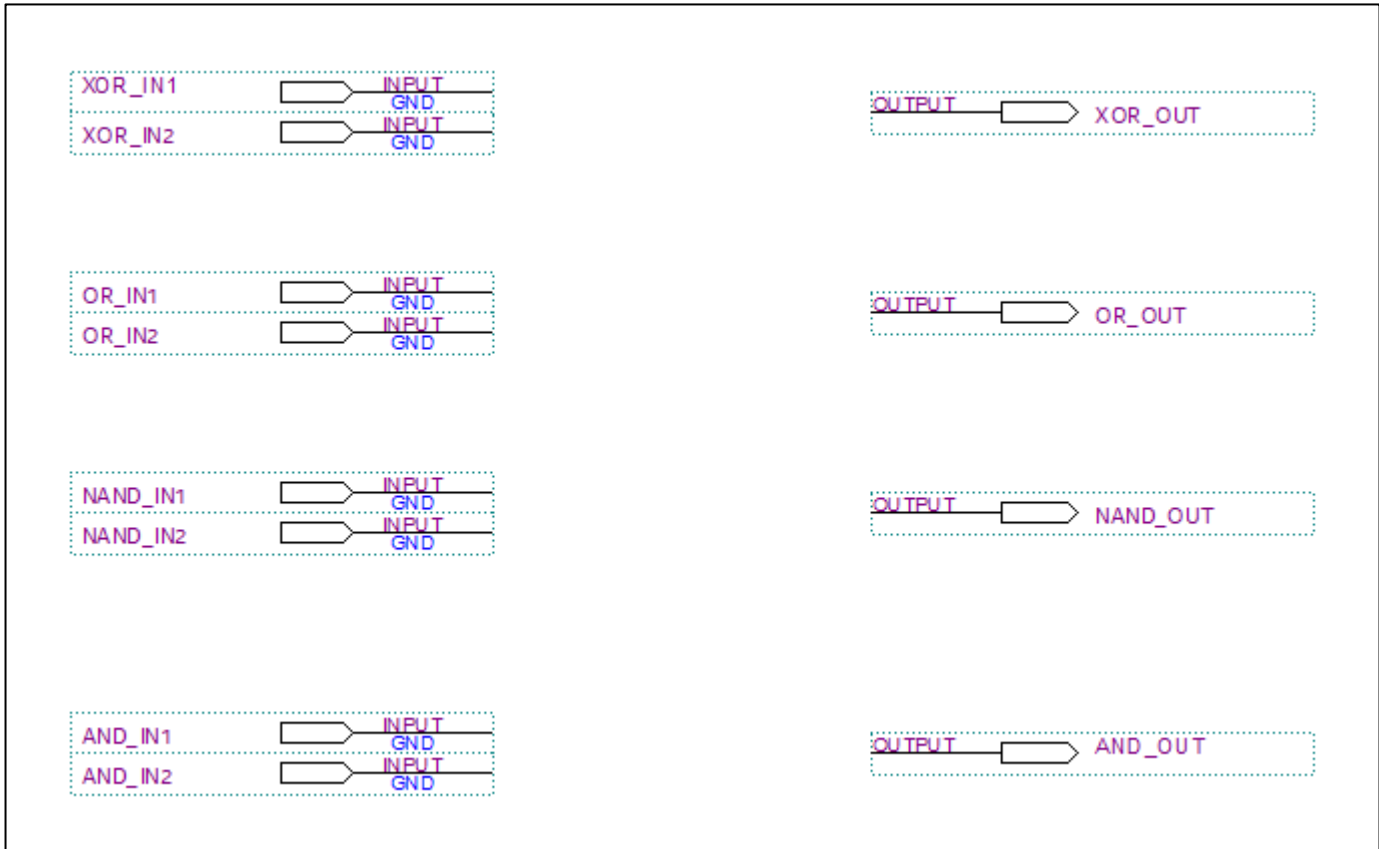
Then in the “schem_gates.bdf” schematic window, there is a toolbar as shown below. Click on the **Symbol Tool** and the Symbol window will appear. Add part symbols to schem_gates.bdf



First, add some input pin symbols to schem_gates.bdf. Within the Symbols Dialog box browse to the Quartus libraries and then SELECT the **Primitives/pin** subfolder. Here you can select the pins as required and place them on the schematic sheet.

Add INPUT pins on the left and OUTPUT pins on the right (typical for schematic convention for design flow).

Add Inputs and Outputs to the **schem_gates.bdf** schematic sheet as shown below:



List of inputs:

XOR_IN1, XOR_IN2, OR_IN1, OR_IN2, NAND_IN1, NAND_IN2, AND_IN1, AND_IN2

List of outputs:

XOR_OUT, OR_OUT, NAND_OUT, AND_OUT

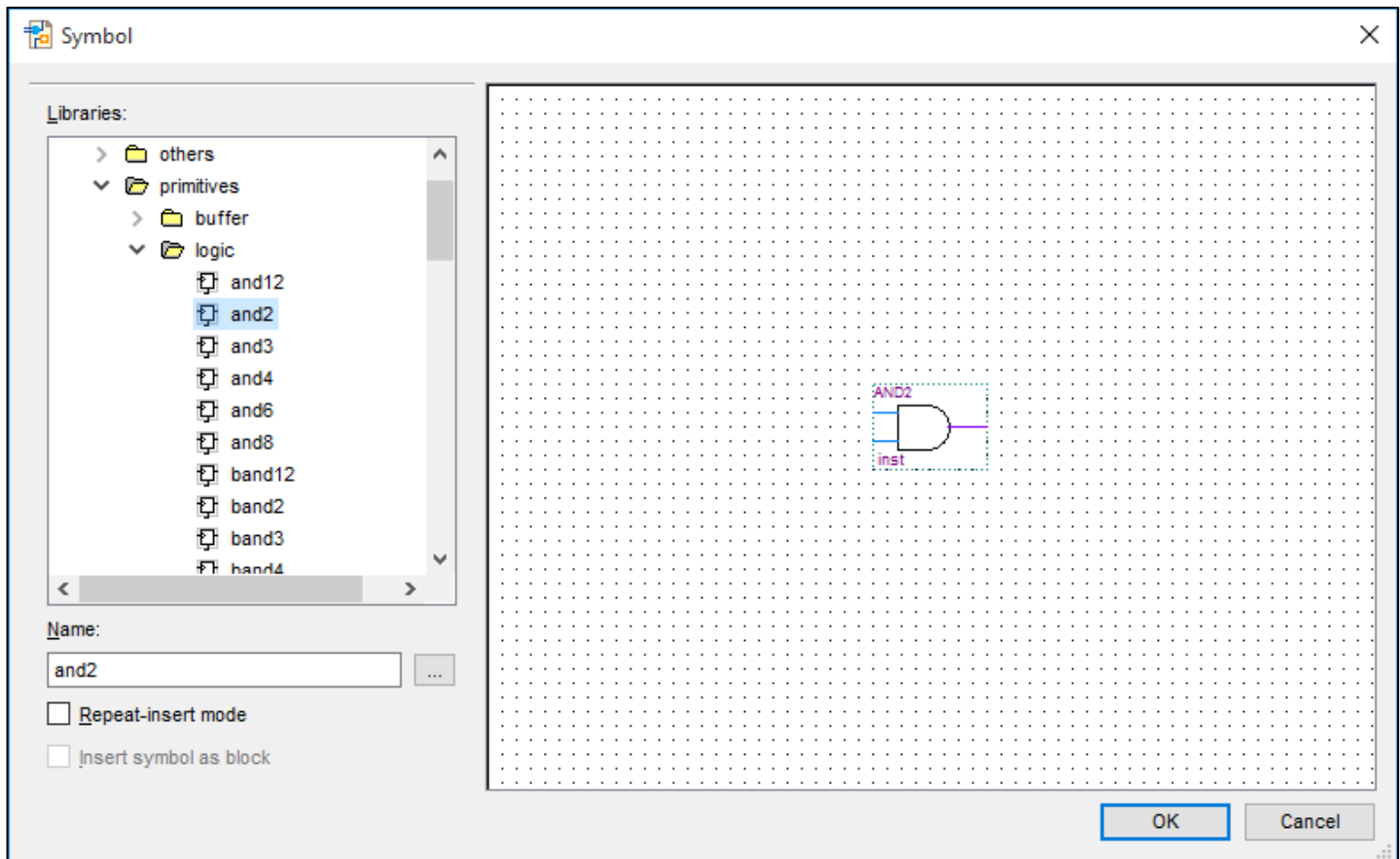
Name each of the pins as in the lists above for this block. (Double-click each pin and modify its Name property).

After placing and naming the pins on the schematic sheet, return again to the symbol Libraries for logic gates in the Primitives/Logic folder.

We will only be using two-input gates for this lab. Below is a truth table for the gate functions.

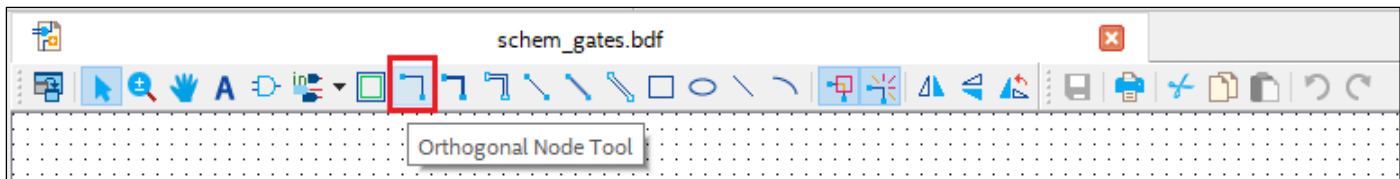
IN 1	IN 0	XOR	OR	NAND	AND
0	0	0	0	1	0
0	1	1	1	1	0
1	0	1	1	1	0
1	1	0	1	0	1
GATE SYMBOL →					

You must locate the basic 2 input gate functions (XOR, OR, NAND, AND) from the Quartus libraries and insert them into this schematic. Below is an example of the two input AND gate.

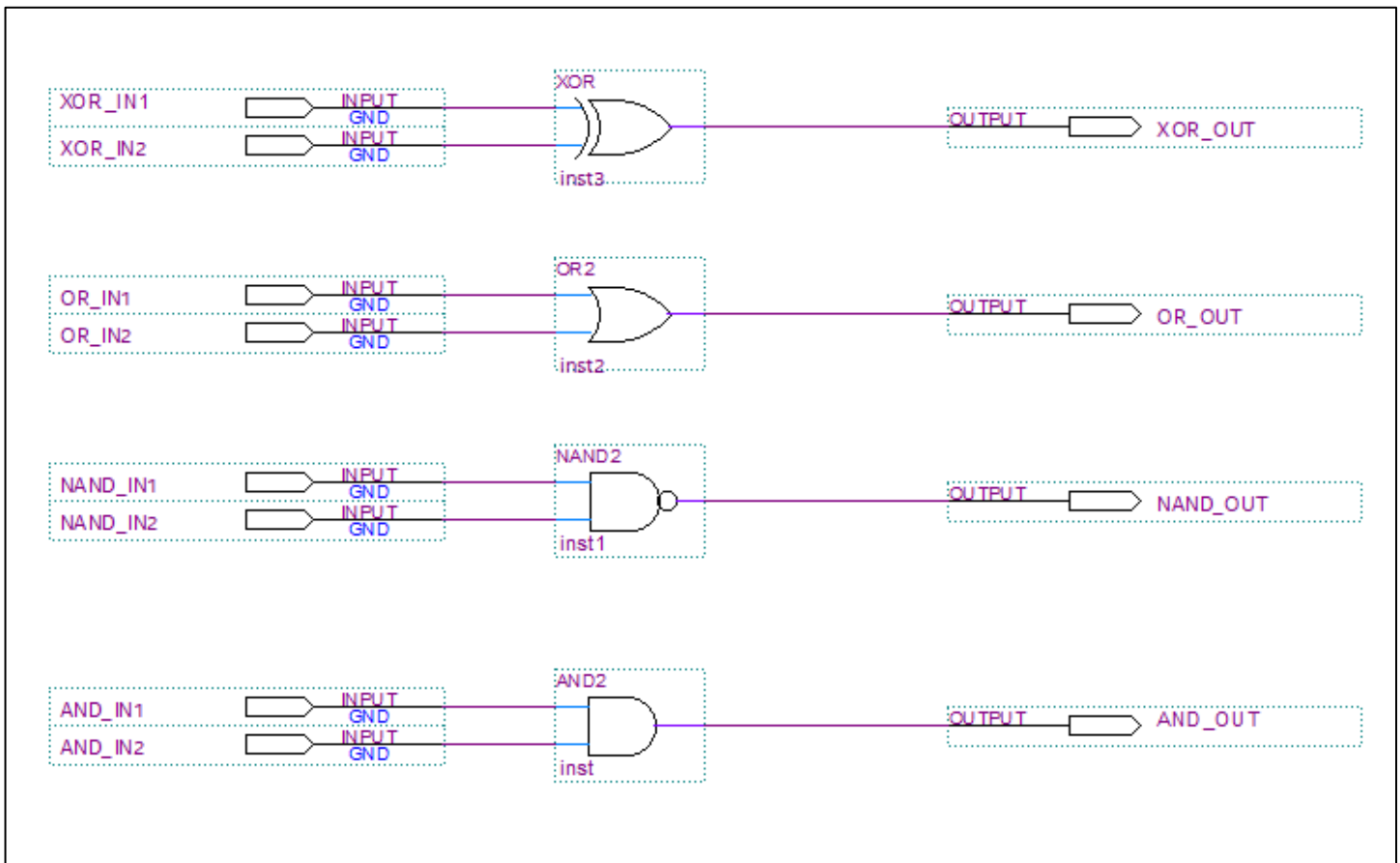


Search for each of the remaining three two-input gates mentioned above and place one of each type on the schematic.

Use the Orthogonal Node Tool (highlighted below).



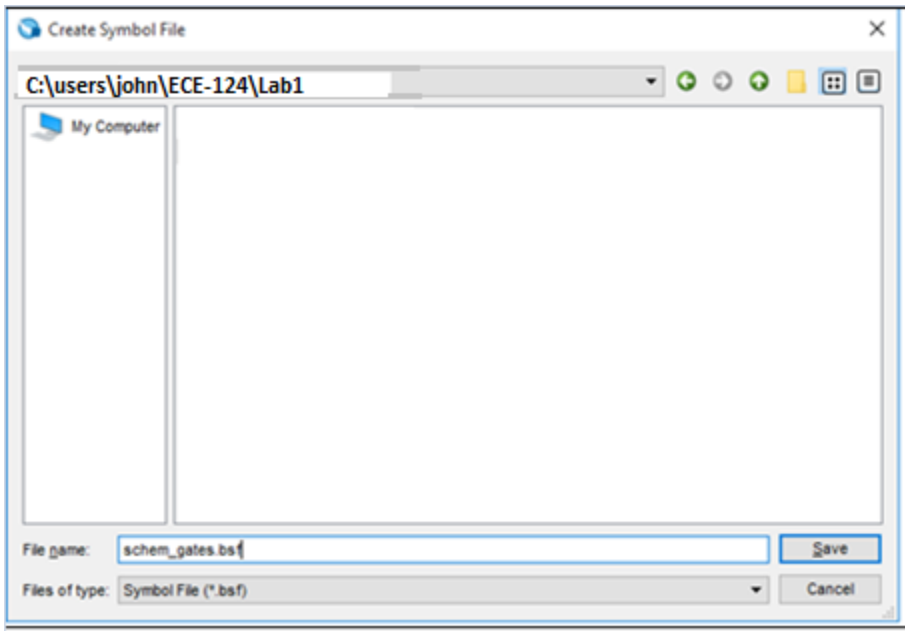
PRESS AND HOLD THE MOUSE BUTTON TO DRAW EACH WIRE. Connect the input pins to the gate inputs and the output pins to the gate outputs as shown below.



After the schematic is drawn, save the file. Go to the FILE Tab and select **File>Save**.

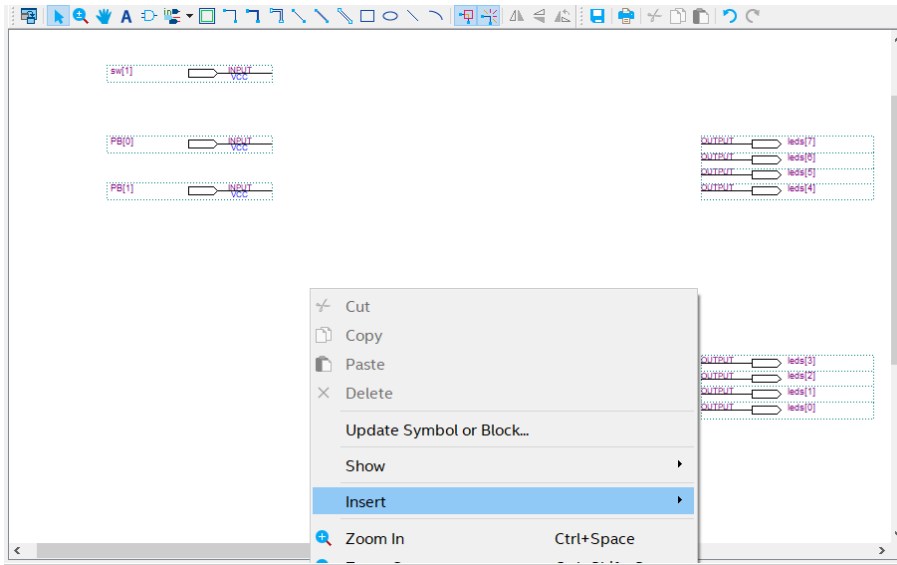
Now that the `schem_gates` schematic design file has been created and saved, we must make a symbol for it so that we can add its symbol to the top-level schematic (`LogicalStep_Lab1_top`). Go to the FILE Tab.

SELECT the **File>Create / Update> Create Symbol Files for Current File** option. A Window like the one below appears. The symbol filename option for “`schem_gates.bsf`” should be visible in the File name field.

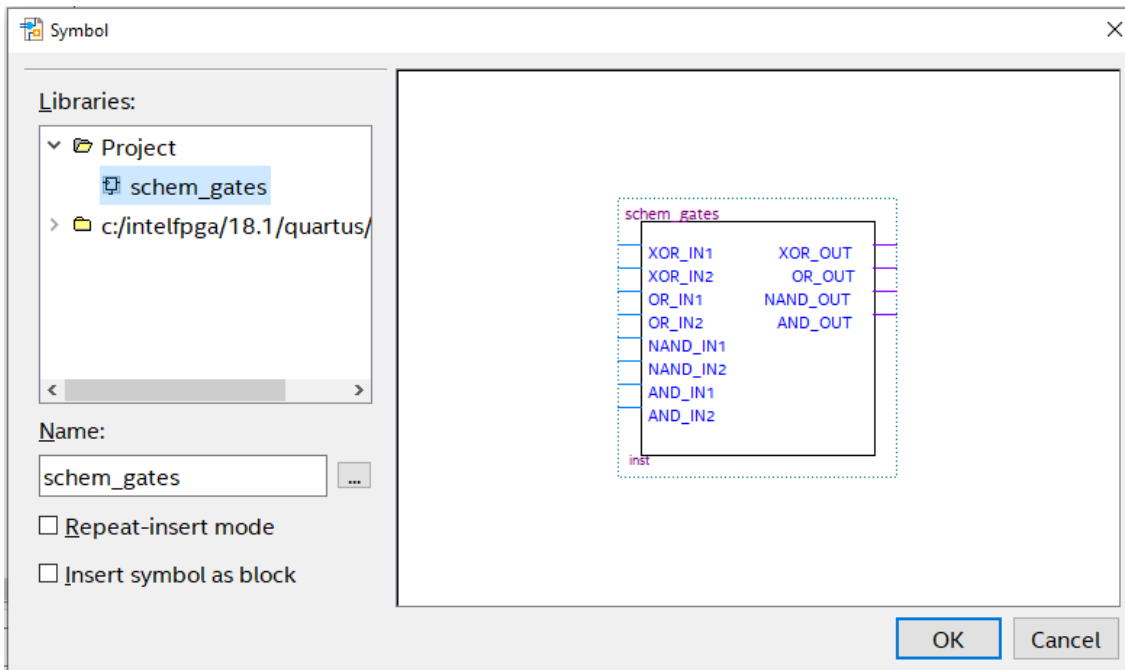


Click **Save**.

Returning to the **LogicalStep_Lab1_top** schematic, insert the new `schem_gates.bsf` symbol (the block that was just created in the previous step). **RIGHT-CLICK** anywhere on the sheet and a pop-up should appear. We wish to add components from the PROJECT library. SELECT the **Insert>Symbol** option.

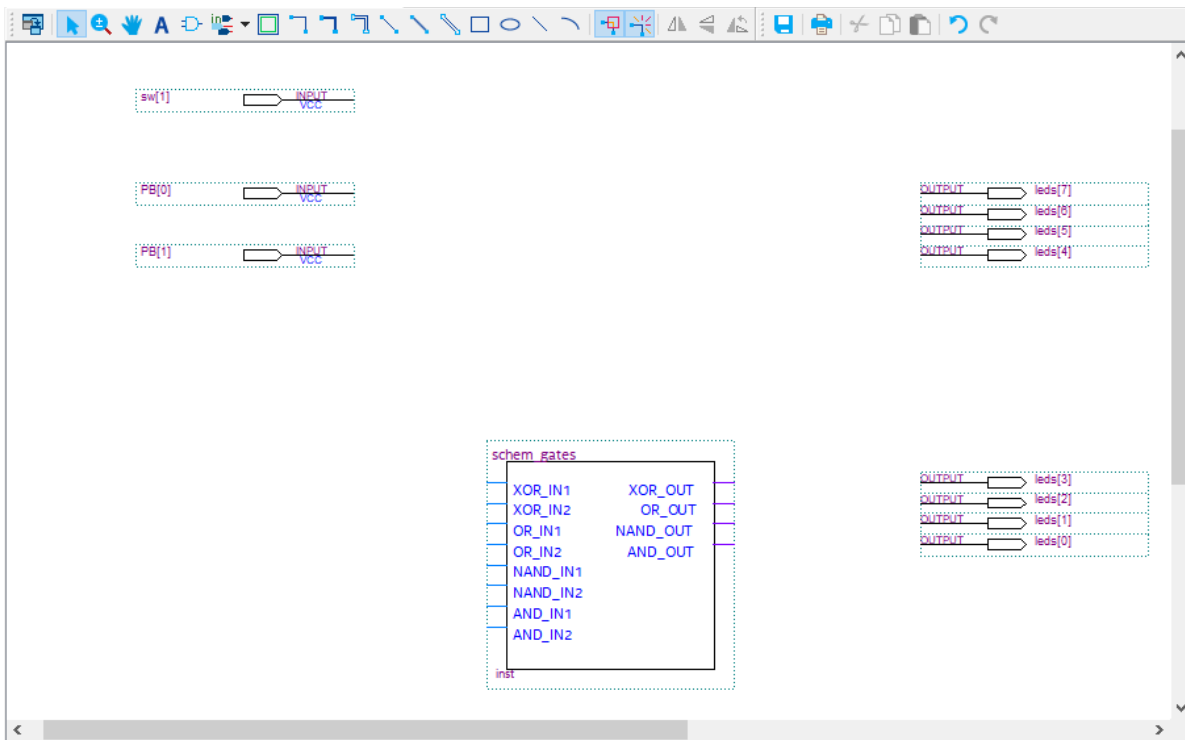


The Symbol Dialog box appears (see below). Expand the **Project** folder and browse to the schem_gates file.

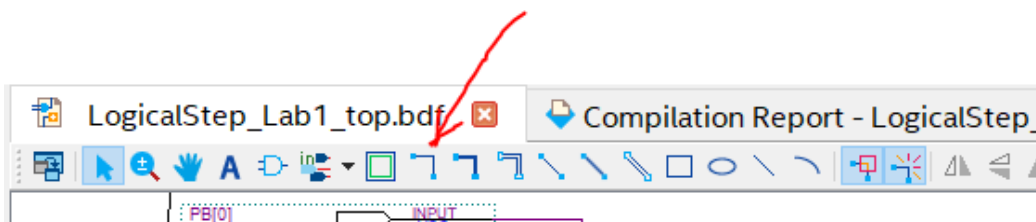


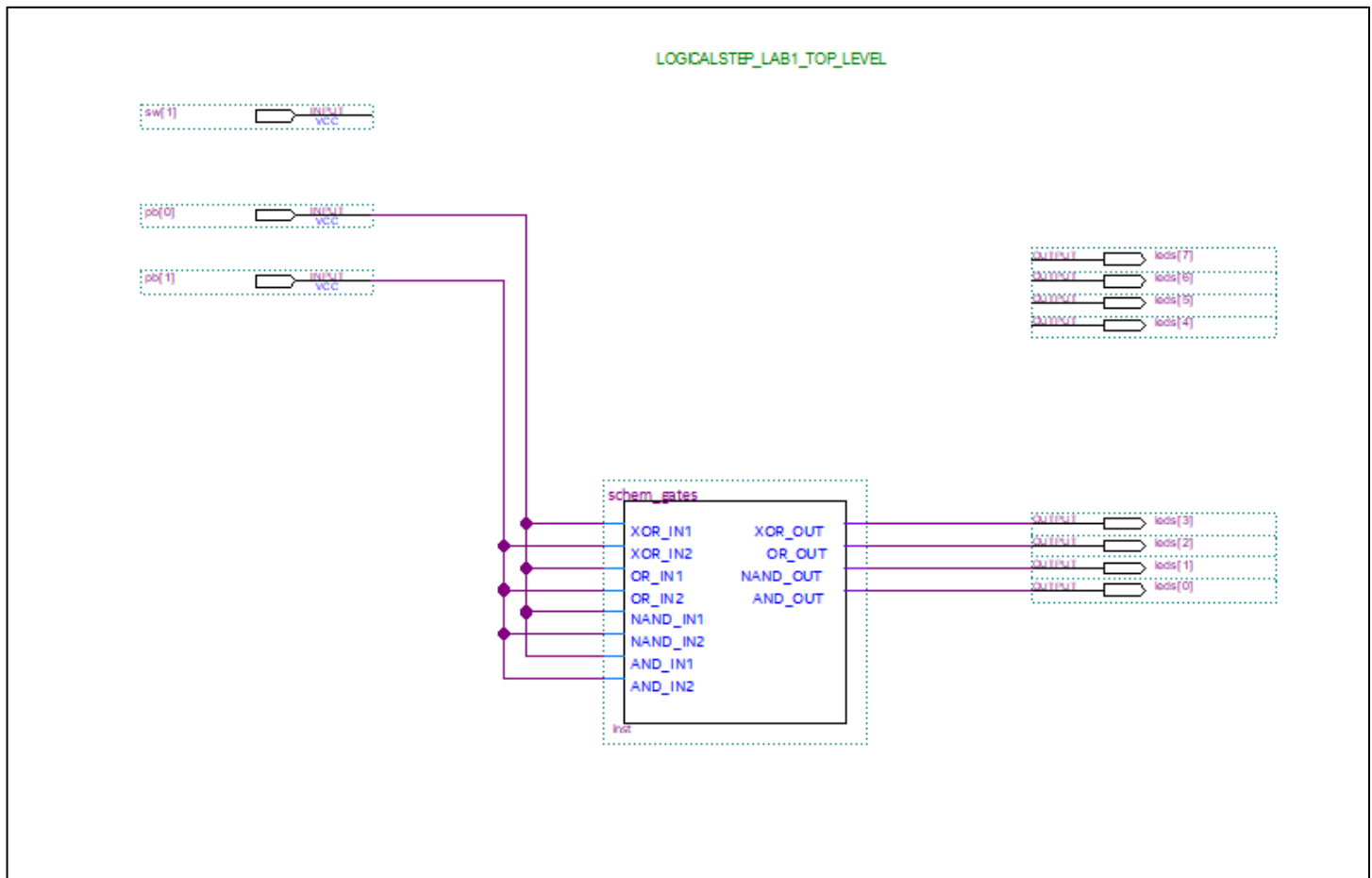
Click the **OK** Button.

Place the symbol on the schematic.



After it is added to the top-level schematic sheet, connect its symbol pins to the INPUT pins and to the OUTPUT pins. Use the ORTHOGONAL NODE TOOL as before (you can hover the mouse over each icon to locate).





SELECT **File>Save** to save the top schematic design shown above.

TAKE NOTE OF WHICH schem_gates functions are connected to which LED pins.

XOR → leds[3],

OR → leds[2],

NAND → leds[1],

AND → leds[0]

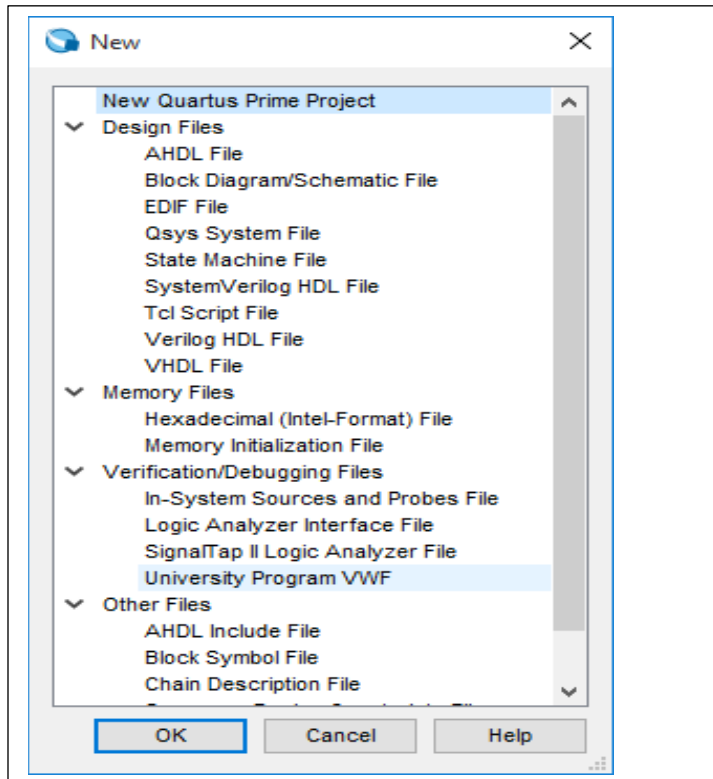
Now we are ready to do some processing of the design and then move into its functional testing by simulation.

2.4.2 Functional Simulations of the Schematic Design

2.4.2.1 Preparation for Functional Simulation – Analysis and Synthesis

Within Quartus go to the PROCESSING Tab. SELECT **Start>Start Analysis and Synthesis** process to process the design details into a gate level logic model that we can use for simulation purposes.

2.4.2.2 Opening a Simulation Window



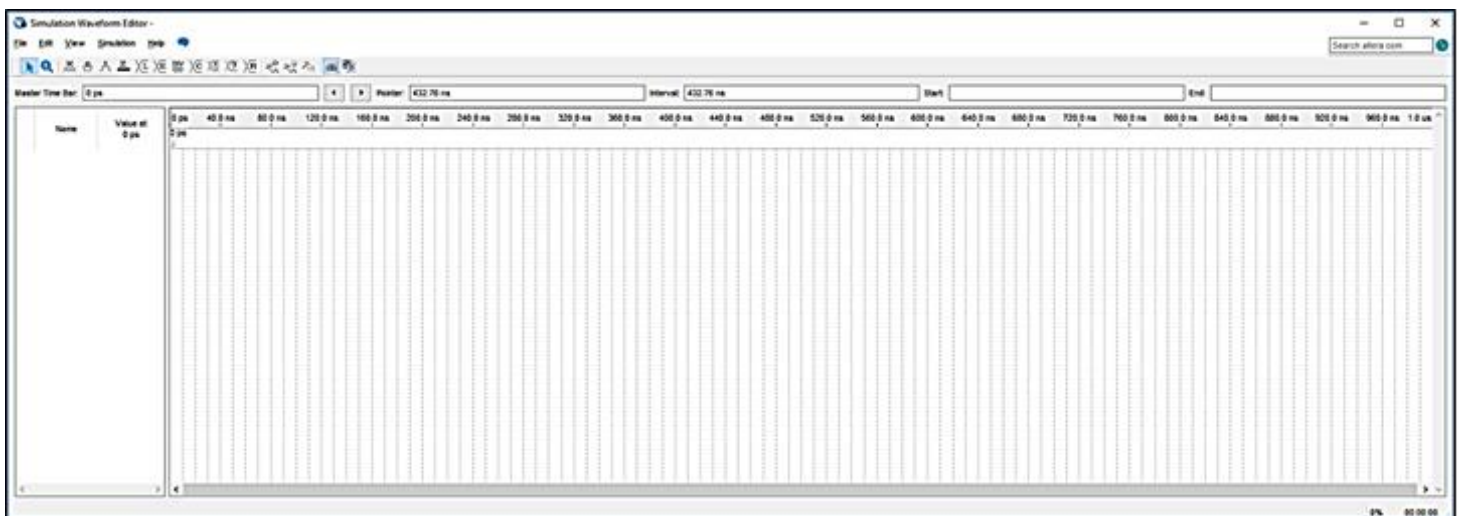
Now under the FILE Tab SELECT the

FILE>New and then SELECT the

University Program VWF utility to open a Functional Simulation. This will be used to illustrate the gate-level functionality in a visual manner. Insert NODES (nets) from the design (see below) for control and observation.

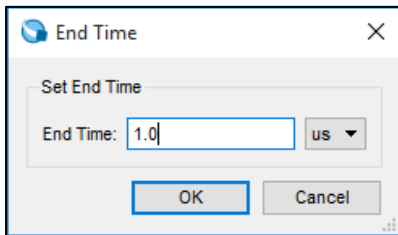
Click **OK**.

A new window will open like the one shown below.



Set the Time scale by going to the Simulator Window EDIT Tab and SELECT the **Edit >End Time** option.

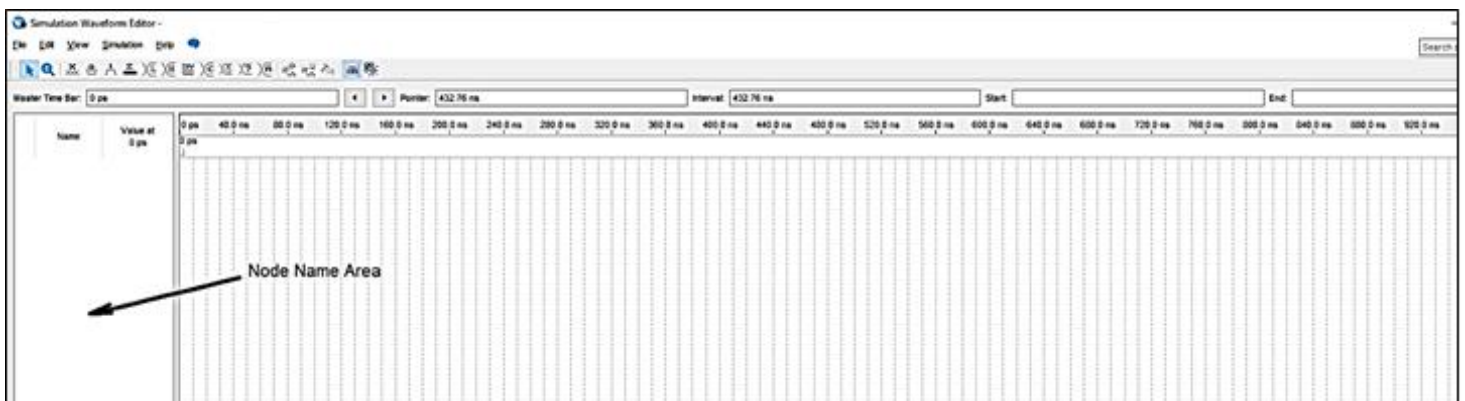
Then a window like the one shown below will appear.



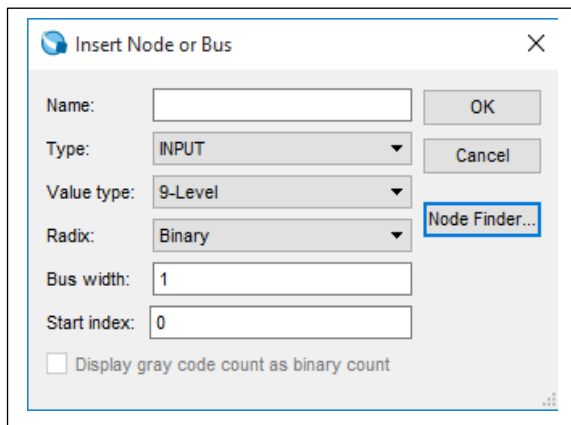
Set it to 1 usec. Click the **OK** button.

2.4.2.3 Adding Nodes to the Simulation Window

For the simulation, only the two Push-Button inputs and the first four LED outputs are required to be inserted into the simulator.

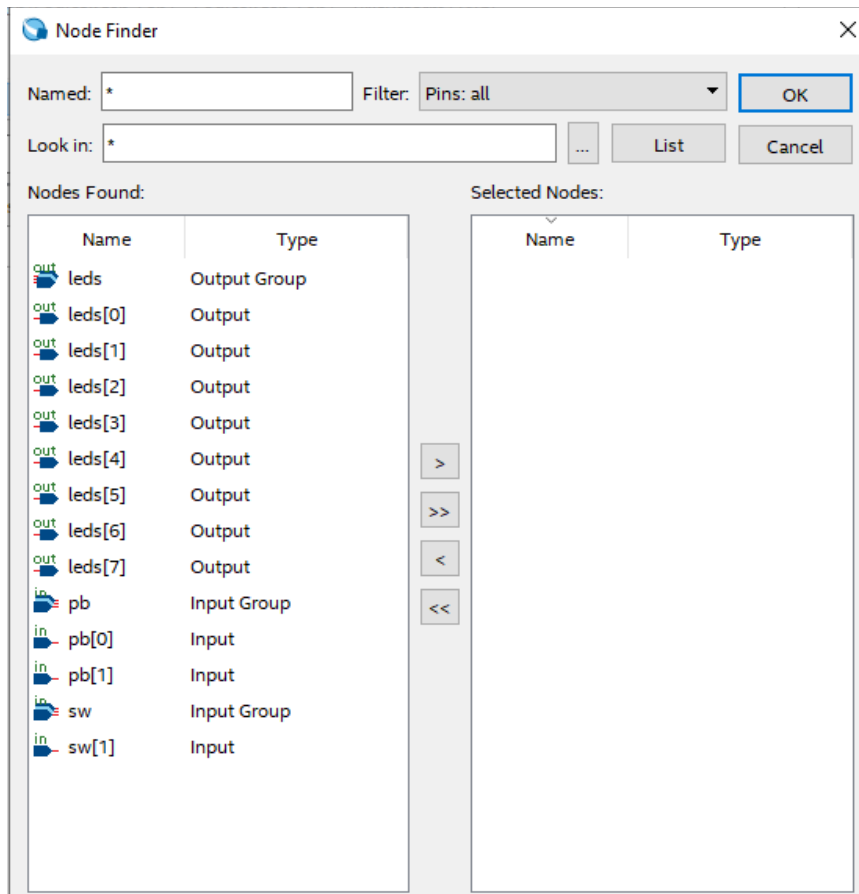


Double-Click the **Node NAME** area of the Simulator Window (shown above) and the following Dialog window will appear as below:



Click on the **Node Finder** Button for faster node identification and insertion. The Node Finder Dialog Box will appear. This will allow you to browse the synthesized design for nodes (nets) to probe for the Functional Simulation.

With the **NODE FILTER** setting set to "**Pins all**" Click on the **LIST** button as shown. The list of pins from the design will appear as shown. ***If you have changed the NAMES of the I/O pins on the LogicalStep_Lab1_top schematic, then your list of pins will look different than shown below.***



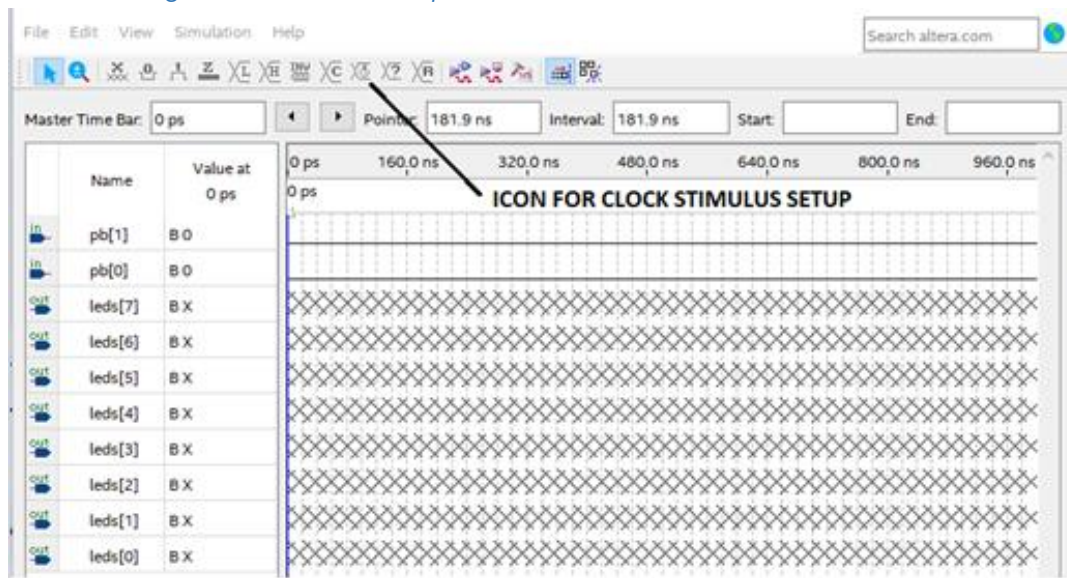
SELECT only the following pins **in the order** specified:

pb[1], pb[0], leds[7], leds[6], leds[5], leds[4], leds[3], leds[2], leds[1], leds[0].

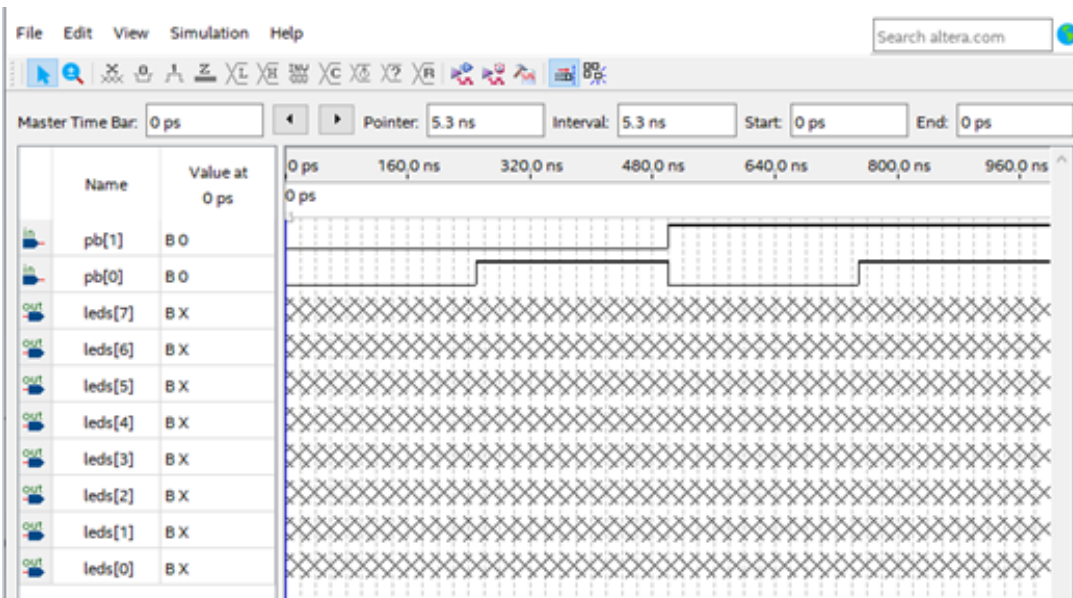
The other pins outside of this list are not yet included in this design.

After selection of the group click on the '>' button to copy them to the Selected Nodes pane. You should ALWAYS select the nodes of groups with the Most-Significant node first (e.g. pb[1] first, then pb[0] etc., leds[7] first then leds[6], leds[5] and so on (**NOTE:** This sequence eliminates getting incorrect values from appearing later simulations for the course when using hexadecimal formats). Then click on the **OK** button and again Click on the **OK** button on the Node_Finder Dialog Box.

2.4.2.4 Adding Stimulus to the Input Nodes



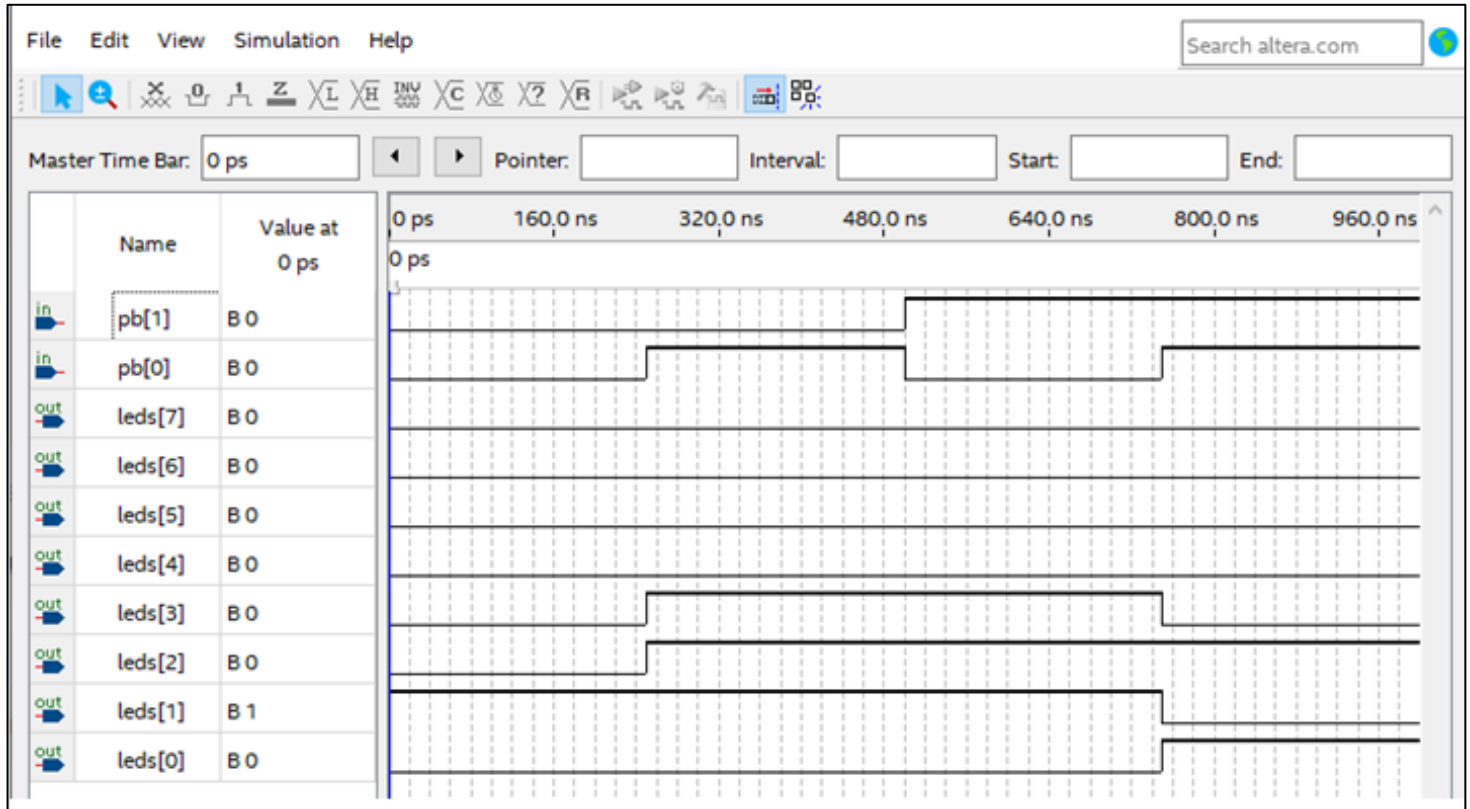
To provide the stimulus waveforms to the input pins SELECT input **pb[0]** in the NAME column, then Click on the **OVERWRITE CLOCK** button (shown above) and enter a period of 500 nanoseconds. Then similarly, for the pb[1] input SELECT the **pb[1]** in the NAME column and then click on the **OVERWRITE CLOCK** button (shown below) and enter 1000 nsec for the period value. These two entries should create waveforms for stimulus as shown.



The stimulus is now created. However, the leds[] outputs are still undefined since the simulation has not yet been run. Save the file as waveform.vwf by going to the Simulator window FILE Tab and SELECT the **File>Save** option.

2.4.2.5 Running the Functional Simulation

In the Simulator window SIMULATION Tab and SELECT the **Simulation>Run Functional Simulation** option. **DO NOT USE the Generate Modelsim Testbench and Script option or the Run Timing Simulation option.** The simulation results should look like the figure below when it has completed.



In the example being used, recall that the leds[0] pin is connected to the AND_OUT pin of schem_gates in the design. Similarly, leds[1] is connected to NAND_OUT, leds[2] with OR_OUT and leds[3] with XOR_OUT. Confirm that the simulation waveforms follow the gate truth tables covered earlier.

Note that there is no simulation activity on the leds[7] downto leds[4] nodes yet.

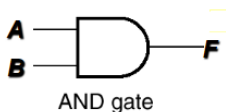
Now you can CLOSE the waveform windows. The Simulation STIMULUS file will be saved in your project folder as waveform.vwf. You will open it directly in steps coming up later in the Lab1 exercises.

2.4.3 Basic Verilog Design Entry Method

The four main areas of focus in the Verilog files for this course.

1. **Module Declaration:** declares the design unit name and the port names associated with it. At the end of the Verilog file there must be an endmodule statement. Notice that the Module Declaration has no information about how the hardware block uses the inputs or how to produce the outputs
2. **Port Declarations:** specifies the actual functionality of the ports used. Each port name, type (input or output) and width (number of bits) is declared in this section. The port declarations may alternatively be defined within the Module Declaration section.
3. **Internal Signal Declarations:** for signals used only WITHIN the module.
4. **Verilog Description/Logic Implementation:** this describes how the module works using the inputs and Verilog operators to generate outputs. Other Verilog modules may be defined within a module for a hierarchical organization.

Below is a starting example of a complete Verilog unit for a two input AND gate:



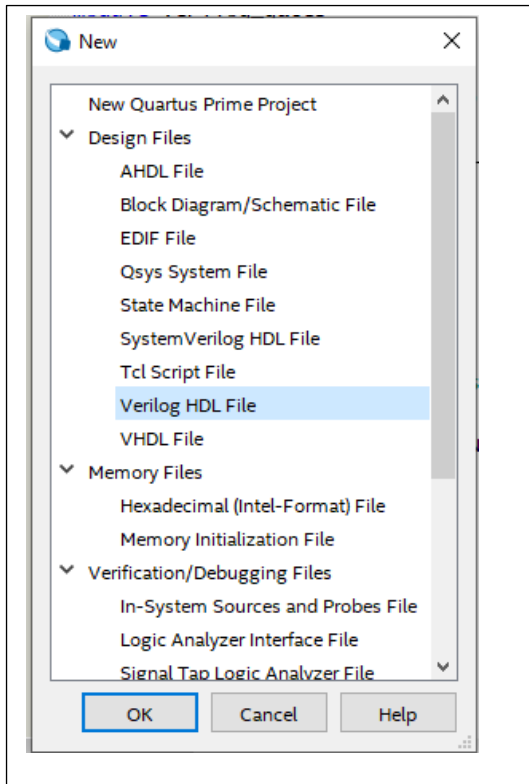
```
module verilog_gates
(
input  A, B,
output F
);
assign F = A & B;
endmodule
```

The “&” symbol above is a reserved operator keyword for Verilog in Quartus.

More examples will follow later on in the course.

The Verilog design entry method within Quartus will now be covered. Similar to how we created the design for the schem_gates block we will now create a Verilog design block.

Returning to the LogicalStep_Lab1_top design in Quartus go to the FILE Tab and Select **File>New**.



SELECT the **Verilog File** option. A blank Verilog window will then open in Quartus.

Save this Verilog file as “verilog_gates.v” by going to the FILE Tab and SELECTING the **File> Save As** option.

This verilog_gates.v file is to be an exact functional replica of the schem_gates circuit that was done earlier (for easier comparison later) but entered with Verilog coding.

You must enter all of the Verilog code shown in the figure below. Then fill in the blanks (logic operators) with Verilog coding for the remaining gate function types (make it the same as “schematic_gates”).

```

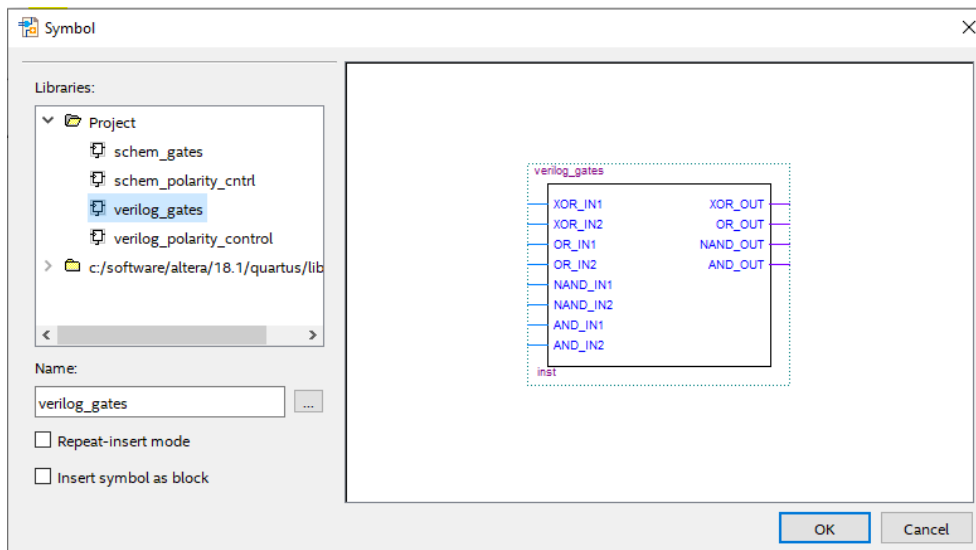
1  module verilog_gates
2
3  (
4      // Input Ports
5      input  XOR_IN1,XOR_IN2,OR_IN1,OR_IN2,NAND_IN1,NAND_IN2,AND_IN1,AND_IN2,
6
7      // Output Ports
8      output XOR_OUT, OR_OUT, NAND_OUT, AND_OUT
9  );
10
11  // verilog logic operators
12  // XOR:      ^
13  // OR:       |
14  // NOR:      ~(.. | ..)
15  // AND:      &
16  // NAND:     ~(.. & ..)
17  // XNOR:     ~(.. ^ ..)
18
19  // fill in the missing operators below:
20
21  assign AND_OUT = AND_IN1 AND_IN2;
22  assign NAND_OUT= ~(NAND_IN1 & NAND_IN2);
23  assign OR_OUT  = OR_IN1 OR_IN2;
24  assign XOR_OUT = XOR_IN1 XOR_IN2;
25
26  endmodule
27

```

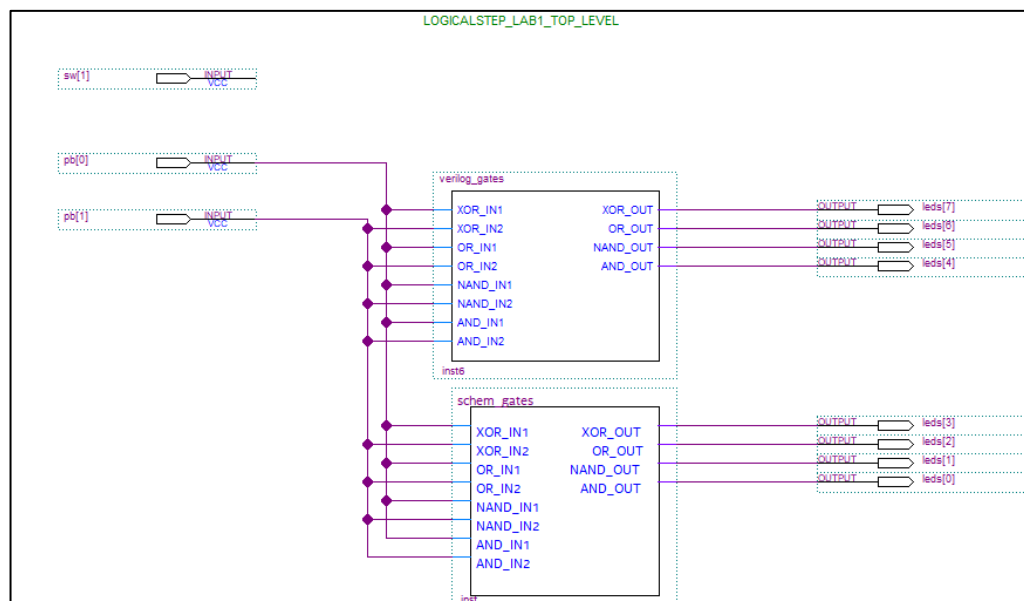
Save the Verilog file by browsing to your **Lab1 project folder** and **Save** the verilog_gates.v file and just leave the file in edit view (to establish it as the current file in Quartus).

Create a schematic block symbol for the verilog_gates design using **File>Create / Update>Create Symbol files for Current File**. **Save** the verilog_gates.bsf symbol.

Go back to the top-level schematic design to insert (“instantiate”) the new verilog_gates symbol as shown below. Recall that to select the symbol **RIGHT-CLICK** anywhere on the top-level schematic and **SELECT** the **Insert>Symbol** option. Browse to the Project folder in the Symbol Window and Select the **verilog_gates** symbol and Click **OK**.



On the schematic, connect the verilog_gates symbol inputs to the same pin INPUT connections as were used for the schematic_gates symbol and connect the verilog_gates block outputs to the LEDS(7) to LEDS(4) output pins as shown below by using the ORTHOGONAL NODE TOOL as before.



Save the design.

Run another **analysis and synthesis compile** of the new FPGA design with **Processing>Start Analysis and Synthesis**. Open the simulation window again (by File>OPEN>waveform.vwf **with file filter set to “ALL FILES”**) and run the simulation again. Confirm that both the schematics and Verilog implementations work the same by observing the respective patterns on the two sets of LEDS outputs (leds3 to 0 for schematics and leds7 to 4 for Verilog gates).

2.5 Adding Some Output Polarity Control

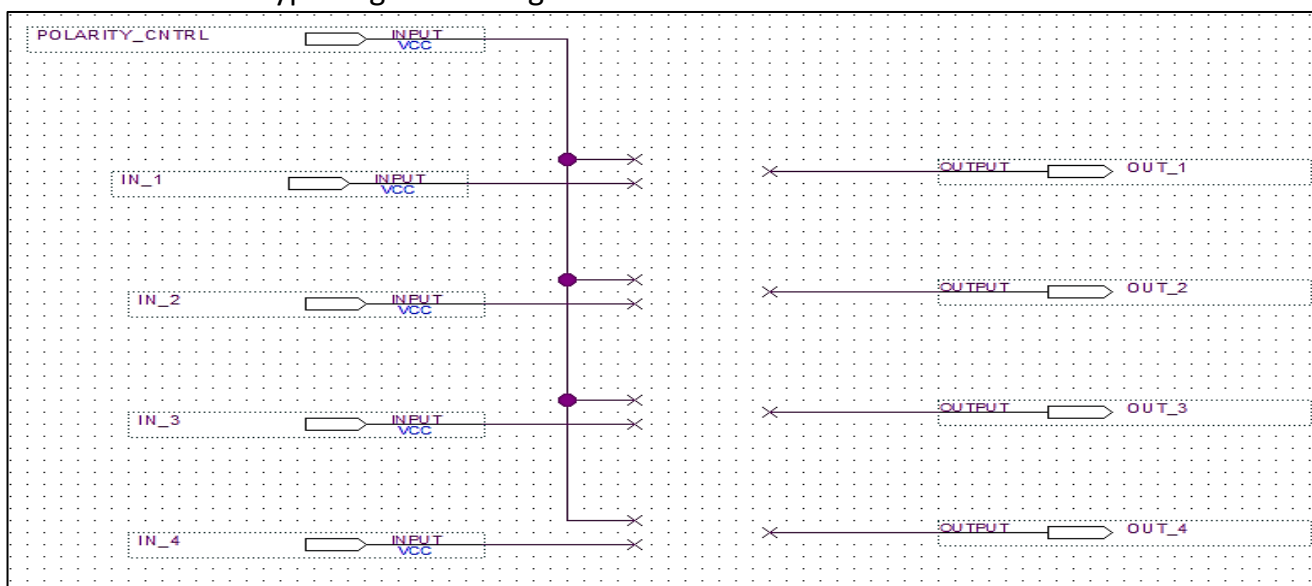
Sometimes digital circuits use the opposite polarity for the **active state**. An **Active-Low polarity** means that the “0” level represents the “ON” state and the “1” level represents the “OFF” state. The way things were created, earlier in this Lab, was always with **Active-High polarity**.

A new block will be added to offer a choice of polarity for the output signals as a whole for LAB1. All of the Lab1 outputs will become Active High when the Polarity Control input is High (e.g.: ‘1’). All of the Lab1 outputs will become Active Low when the Polarity Control input is Low (e.g.: ‘0’).

Your **Lab1 design** must now add another two blocks (one in schematic form and one Verilog design form) that allows the **sw[1]** input to be used as an output polarity control for all of the output pins.

Within each of these two blocks the Polarity Control function must use a **single common 2 pin gate type per path**. One input of each gate will be connected to the common Polarity Control input pin and the second gate input will be connected to the input pin as shown below.

Hint: Create a gate truth table for various types of two-input gates from the library and determine what type of two-input gate to use. For the schematic version, the connections are shown below but with the common type of gate missing.



For the Verilog version, you may use the following info below.

```

module verilog_polarity_control
(
    // Input Ports
    input POLARITY_HIGH, IN1, IN2, IN3, IN4,
    // Output Ports
    output OUT1, OUT2, OUT3, OUT4
);

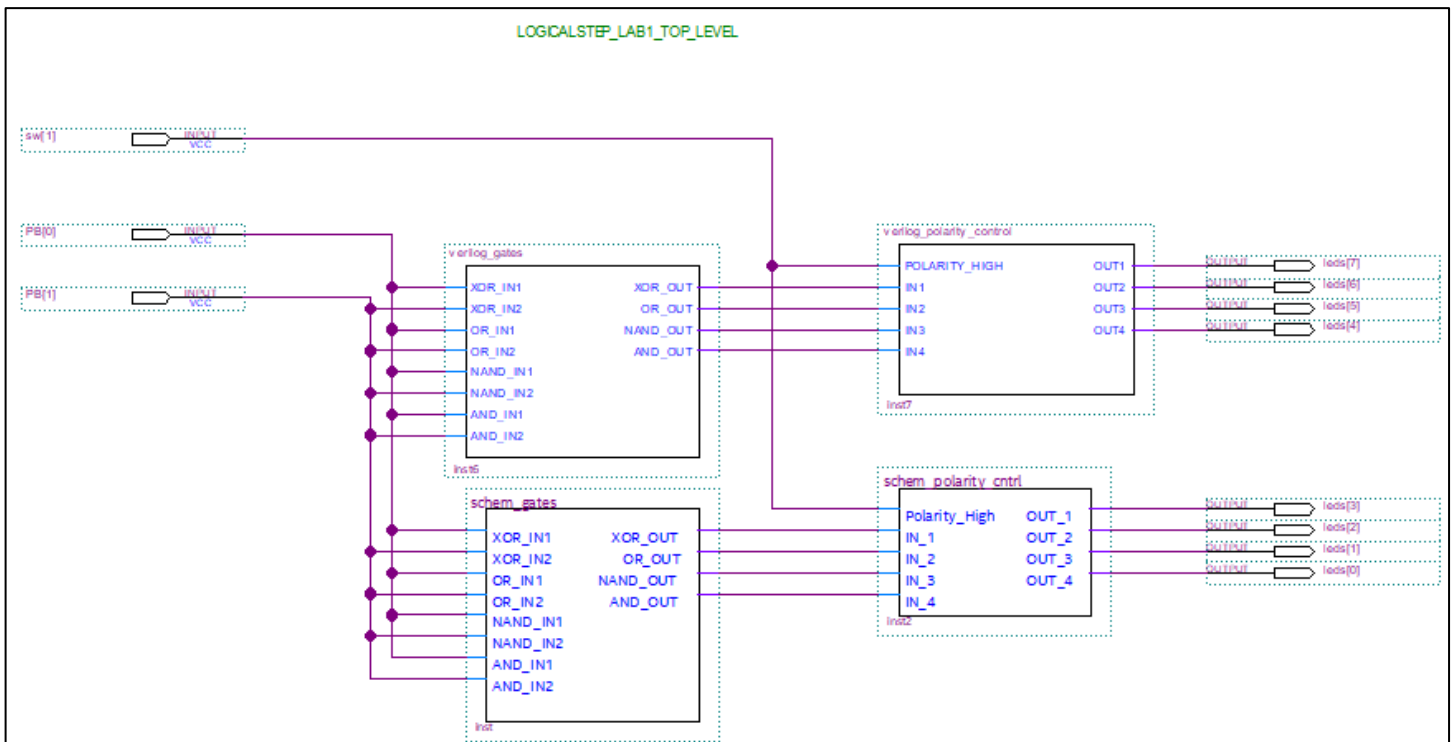
    assign OUT1 = POLARITY_HIGH IN1;
    assign OUT2 = POLARITY_HIGH IN2;
    assign OUT3 = POLARITY_HIGH IN3;
    assign OUT4 = POLARITY_HIGH IN4;

endmodule

```

For the Verilog version, **make sure that you save that file with the same name as the name used in the Module Declaration.** Each of the OUT1 to OUT4 functions must have a logic expression of the common POLARITY_HIGH signal and the corresponding IN1 to IN4 input.

Create, save and add a schematic block symbol for EACH of your new blocks as before (using **File>Create / Update>Create Symbol files for Current File**) and add it to the LogicalStep_Lab1_top schematic with the required connections mentioned above.



Your updated schematic design will look something like the one above.

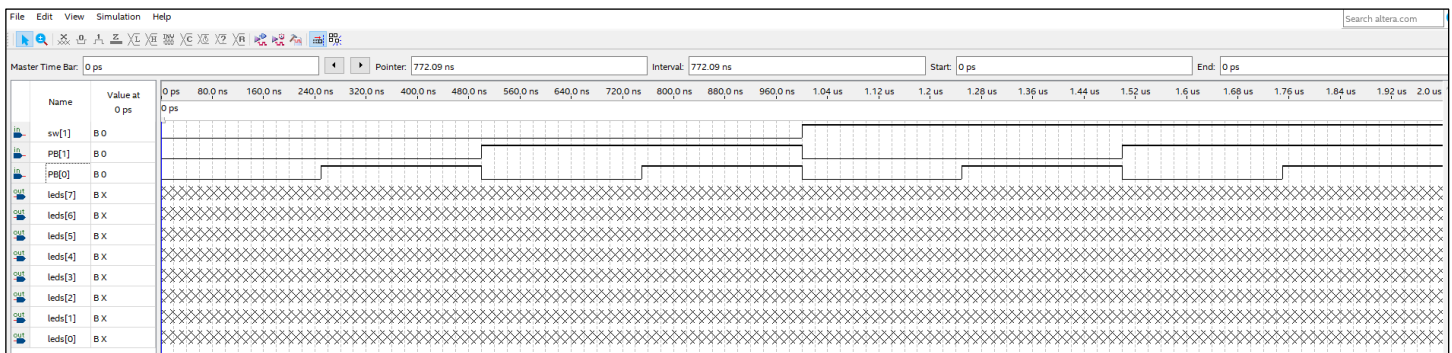
Inputs- pb0, pb1, SW1; Outputs – leds(3 downto 0), leds(7 downto 4);

Run another **analysis and synthesis compile** of the new FPGA design with **Processing>Start Analysis and Synthesis**. Open the simulation window again (by File>OPEN>waveform.vwf **with file filter set to “ALL FILES”**).

Add a new node (SW1) to the Nodes Names area.

Set the simulation's End time is 2us.

Use the following stimulus for the functional simulation of your top-level design with the polarity control feature. The period of sw[1], pb[1], and pb[0] is 2us, 1us, and 500ns respectively.



Take note of the active state of the output pins for each setting of the SW1 Polarity Control switch. Notice how the Polarity_Control changes all of the outputs **active state**. The functionality of the schem_gates and verilog_gates logic blocks is the same as before. BUT the **active state** at the project outputs (at the LEDS pins) can be changed with just a single Polarity Control input setting using SW1.

2.6 Getting Your Lab1 Design into the FPGA on the LogicalStep Board

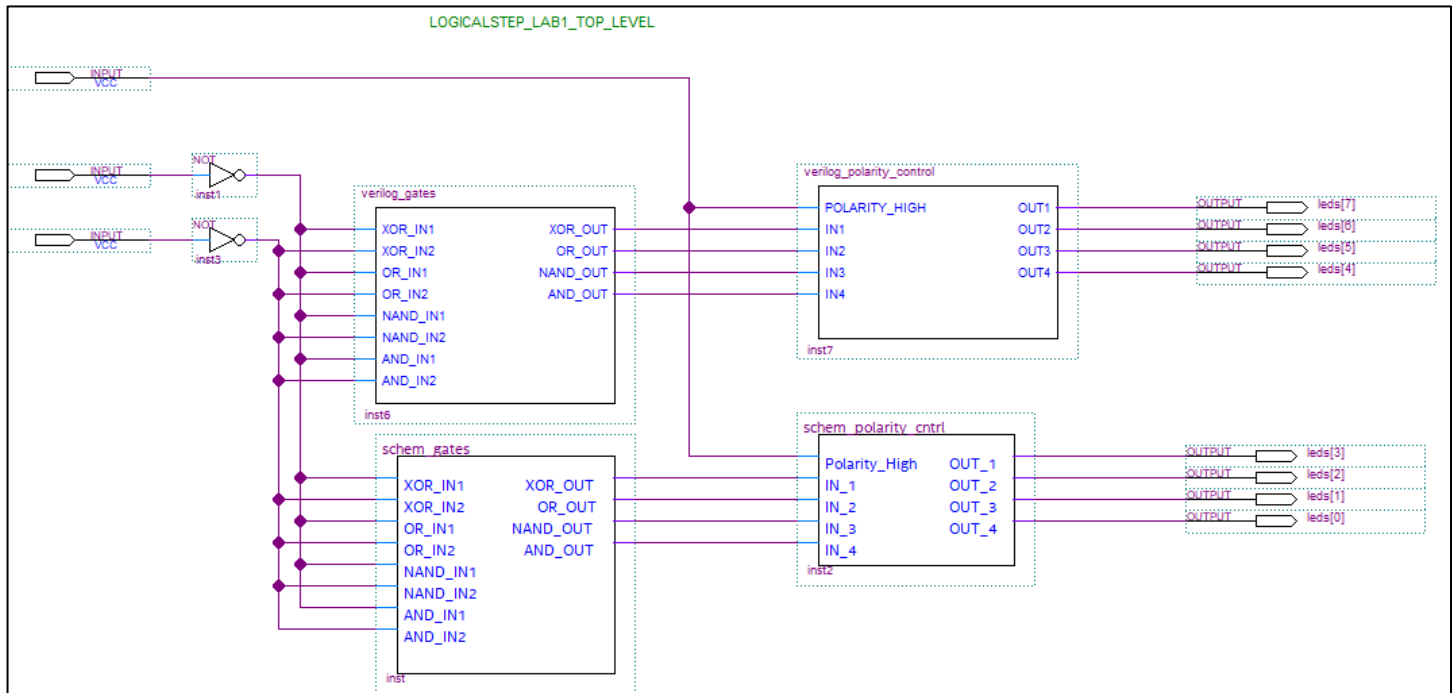
Now, after simulations have proven the functionality of your Lab1 design we can start to get it ready for operation in the actual FPGA on the LogicalStep board.

We must consider the actual signals connected to the FPGA on the LogicalStep board. The discussions above, having to do with ACTIVE STATE, are appropriate. The previous exercises for design entry, synthesis and simulation phases, presented all inputs as **ACTIVE HIGH** for easier understanding.

But, the two Push-Button inputs (pb[0], pb[1]) on the LogicalStep Board are **actually ACTIVE LOW !!**

On the LogicalStep board, when each pb key is pressed the pb signal state is '0' for a "button pressed" condition. But we want a logical '1' to arrive at the appropriate schem_gates block input when the pb key is pressed. So, for Lab1, we must add inverters between those pins and your design logic to convert the ACTIVE LOW input signals to ACTIVE HIGH to maintain the Lab1 design functionality.

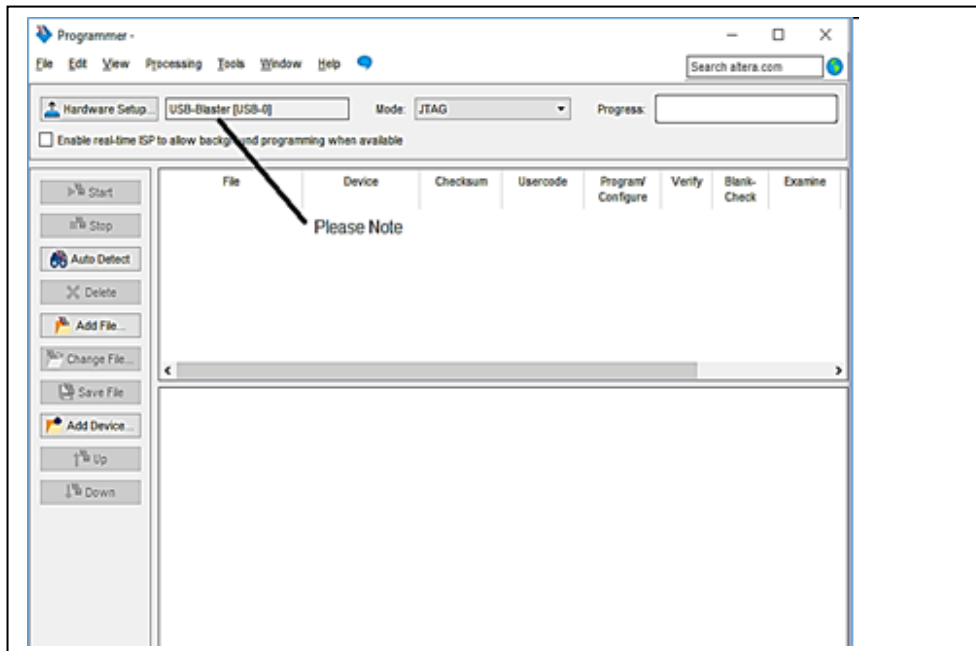
Go to the Quartus libraries again to insert the “not” gate from the Primitives/logic folder for the inverters. Insert and connect them as shown in the figure below.



Now things are ready in the design for a “Design FITTING” compilation for a download to the LogicalStep Board FPGA.

2.6.1 FPGA Design Compilation for Download to the LogicalStep (with the Quartus Programmer)

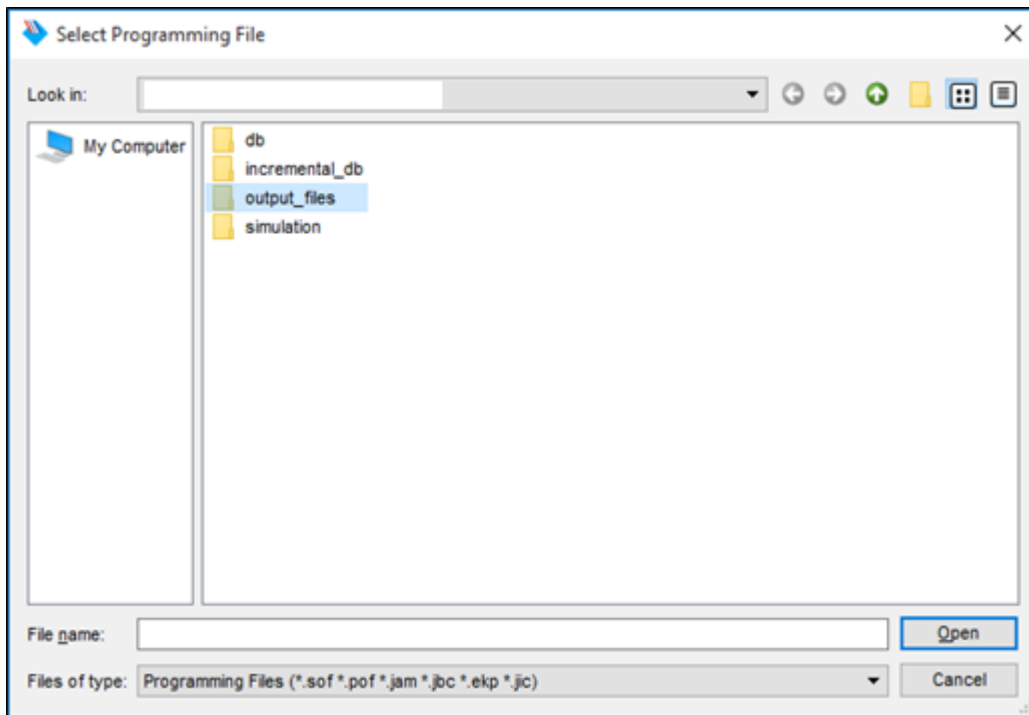
Now the FPGA Compilation process will be executed. Go to the PROCESSING Tab and SELECT the **Processing>Start Compilation** option. When the FPGA compilation finishes, without compilation errors (you may ignore any warnings), then an FPGA load file can be downloaded into the FPGA.



Use the Quartus Programmer to download your design into the LogicalStep board FPGA by going to the TOOLS Tab and SELECT the **Tools>Programmer** option. A Programmer dialog window will open as shown at left. Click the **ADD File** button.

NOTE: If the LogicalStep board is connected, the **USB Blaster** should be seen beside the Hardware Setup field (otherwise speak with the Instructor).

A Select Programming File window will open (shown below). Browse to the **output_files** folder.



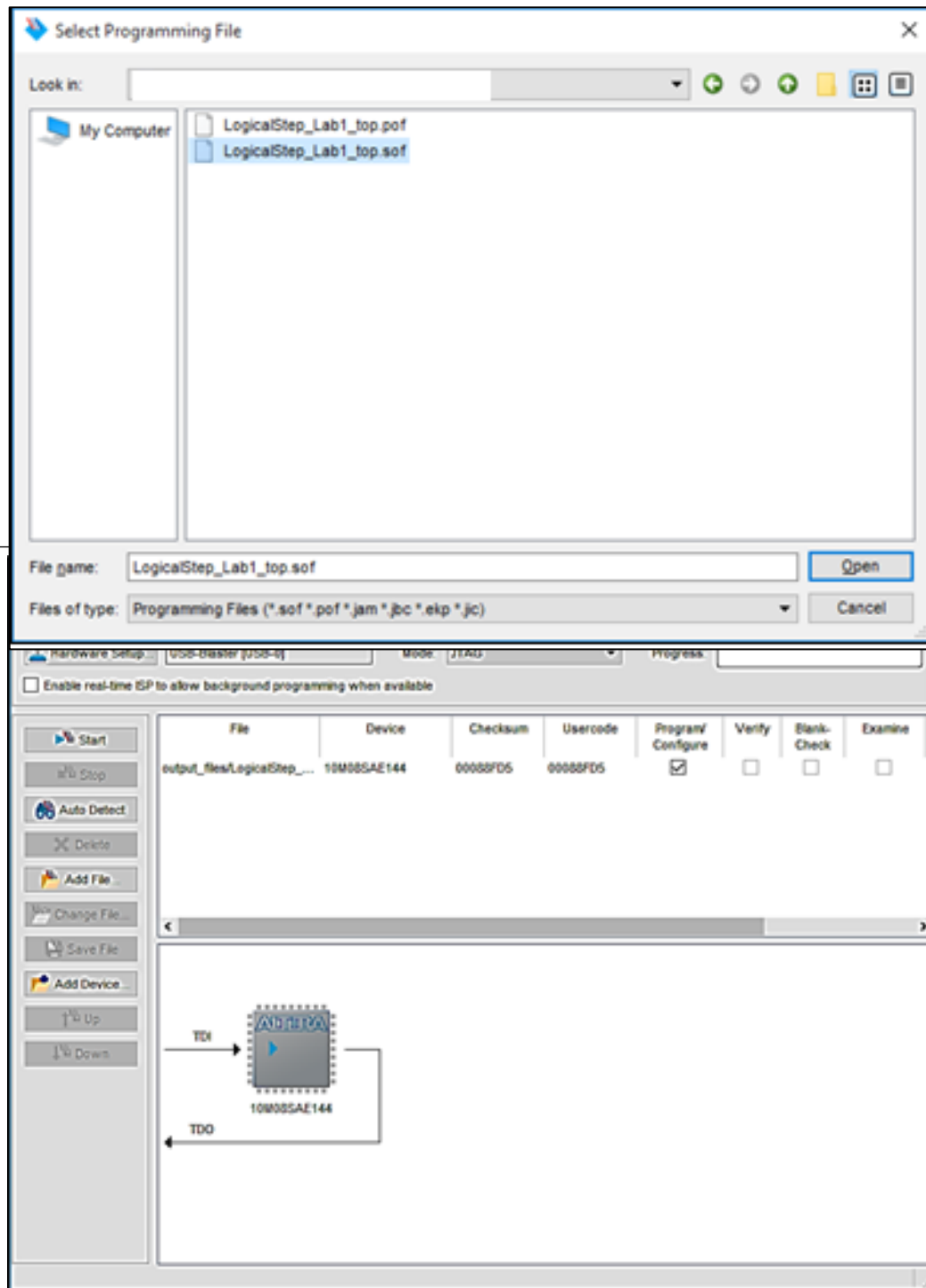
Select the **LogicalStep_Lab1_top.sof file** as shown below.

WATCH OUT!**(NOT the .pof file).**Click **Open**.

The requested file will then show up in the Programmer window as shown below.

Click on the **START** button to begin the FPGA download. The progress window should indicate 100% after the download is complete.

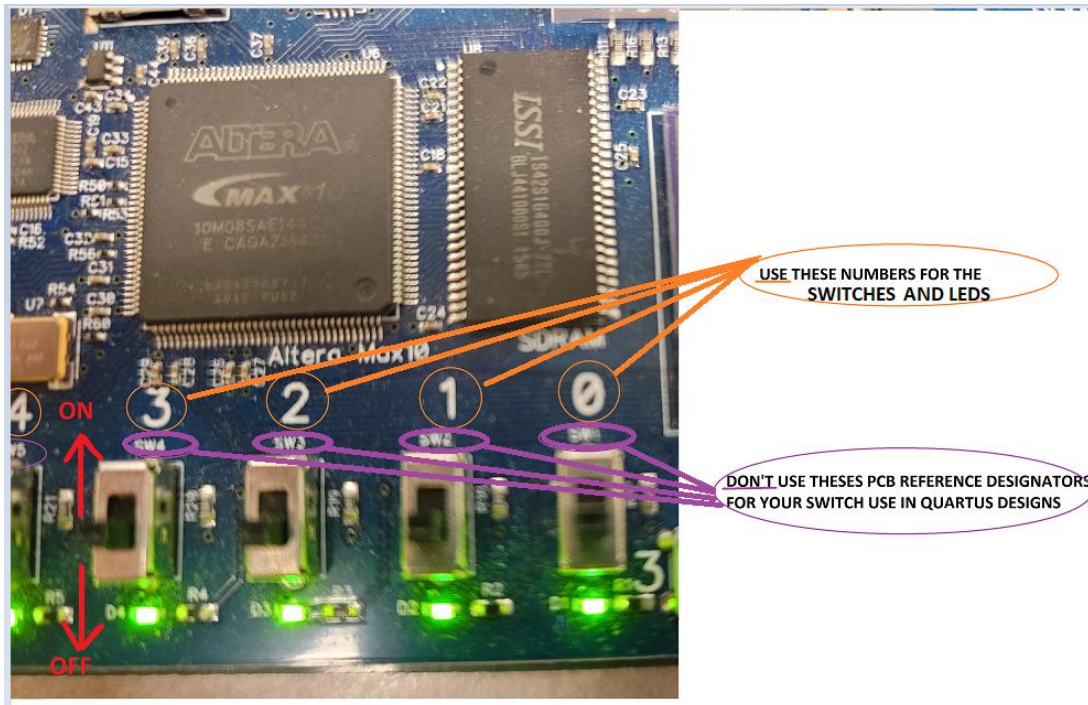
After downloading you



can test your FPGA design by using the pb [1..0] Keys and observing the LED outputs according to your logic gate truth tables. When your work is completed, make sure everything is saved to the Lab1 project folder in your Lab Project folder on the C:\users\xxxxx\ECE-124\Lab1 directory. Then, in the Quartus utility, select the **FILE>Close Project** option and exit Quartus. Then ZIP the Lab1 folder and copy it for a backup to the N: Drive.

When you work on your project design again, copy the zipped folder from the N:Drive to the C:\users\xxxx\ECE-124\Lab1 directory on the workstation where you are working. Complete the new work as before, save everything, then leave Quartus and then zip the folder and copy it back to the N:Drive. The next time that you come to do work in the Lab you will download the project folder from the N: Drive to the C: Drive of the workstation where you are working.

STUDENTS ARE RESPONSIBLE FOR THEIR OWN LAB PROJECT DATA.



2.7 POST LAB1 ACTIVITIES

Your final Lab1 work is below:

1. Complete the testing work for the Lab1 design on the LogicalStep board.
2. Prepare to do a Lab1 DEMO during the Lab2 session. A rubric for the Lab1 Demo is on Learn.
3. A short Quiz of the Lab1 design content, is to be completed during the Lab2 session also.

NEXT LAB SESSION: Now that you have explored some things about the Quartus development tools we will get into a more challenging logic design where we learn and build functions that are used in simple logic designs today.